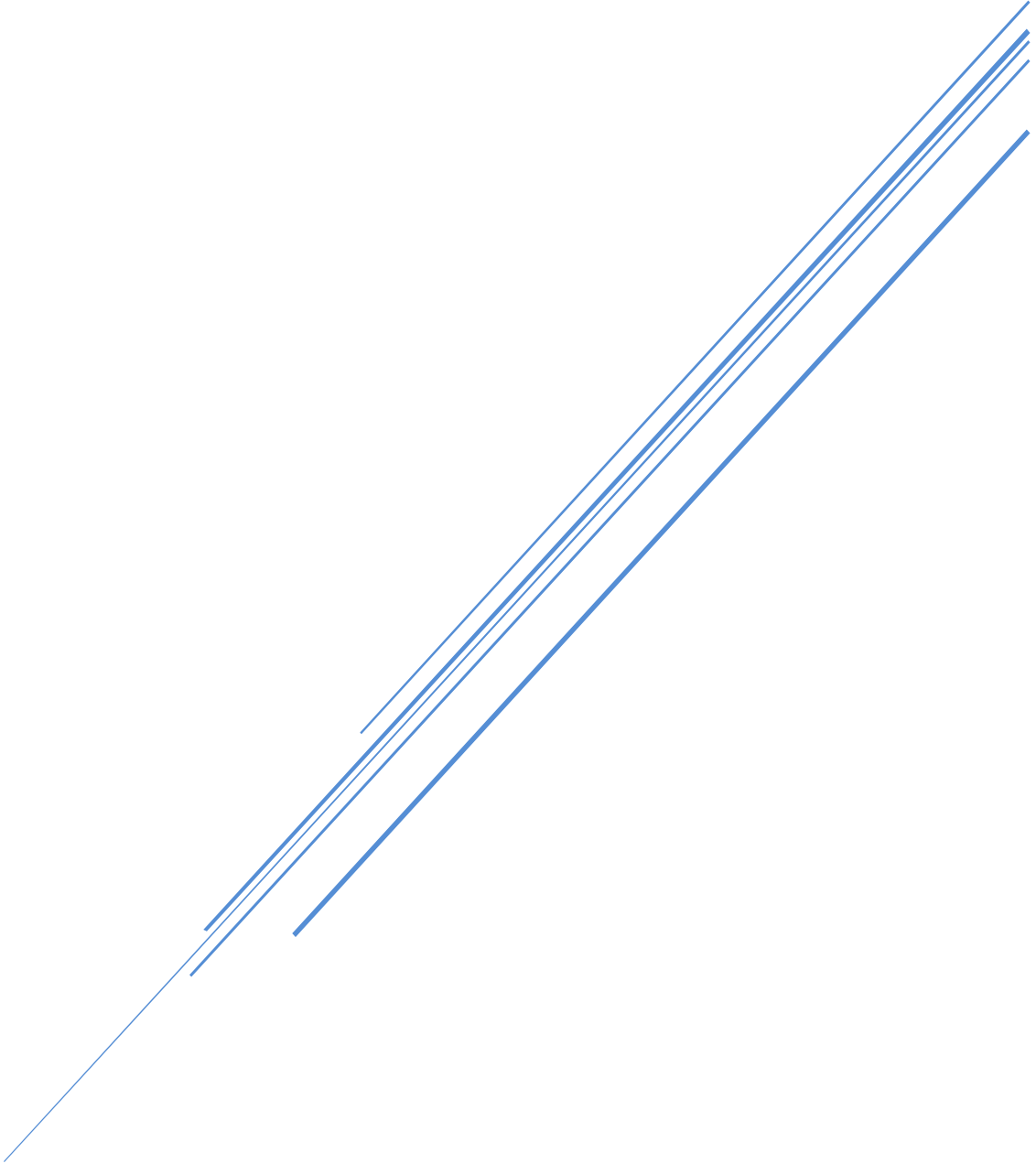


IRONCLAD ARENA

Het Ontwikkelen Van Een Digital Trading Card Game



Marwah Al-Shakhli
Pavel Deleersnyder
Egon Van den Buys

Toegepaste informatica
2025 - 2026

Woord vooraf

Wij kozen dit onderzoeksonderwerp omdat het nauw aansluit bij onze interesses en voorkennis, maar ons tegelijk de kans bood om nieuwe technologieën te verkennen. We hebben al ruime ervaring met REST API's en beperkte ervaring met event brokers, maar een eventbus, een technologie die vaak wordt toegepast in online games, hadden we nog nauwelijks onderzocht. Dit project stelde ons in staat om ons hierin verder te verdiepen.

Daarnaast brengt een online kaartspel een groot en complex domein met zich mee. Het uitwerken van de domain logica beschouwen we als een van de meest interessante aspecten van softwareontwikkeling. Daarom kozen we bewust voor een project dat sterk de nadruk hierop legt.

Ook de keuze van clienttechnologie sprak ons aan, omdat de context van een online game ruimte biedt voor technologieën die in andere toepassingen minder vanzelfsprekend zijn, zoals een game engine. Tot slot vereist een game een sterke focus op gebruikservaring en client design, wat ons de kans gaf om onze kennis van ons vak user experience engineering toe te passen in een nieuwe context.

Deze thesis richt zich tot iedereen die wil begrijpen hoe *Ironclad Arena*, onze eigen interpretatie van kaartspellen zoals *HearthStone* en *Yu-Gi-Oh!*, tot stand is gekomen en welke ontwerpkeuzes en denkwijzen aan de basis van het project liggen. Tijdens het schrijven van deze thesis heeft ieder van ons bijgedragen aan de onderdelen die het meest aansloten bij zijn of haar interesses. Omdat we een project kozen dat ons sterk motiveerde, ervaaarden we het onderzoeksproces als bijzonder leerrijk en kregen we gaandeweg een steeds duidelijker beeld van hoe het spel concreet vorm zou krijgen en welke mogelijkheden en beperkingen daarbij een rol speelden.

Graag willen wij tot slot onze oprechte dank uitspreken aan Dhr. Blomme, Dhr. Casier, Dhr. Debou, Dhr. De Wael, Mevr. Terryn en Dhr. Vlummens voor hun begeleiding, feedback en ondersteuning tijdens dit project.

Abstract Nederlands

Het doel van dit onderzoek is de ontwikkeling van een digitaal kaartspel dat online gespeeld kan worden, waarbij spelers hun eigen kaarten kiezen en strijden tegen andere spelers. Bij overwinningen krijgt de speler nieuwe kaarten.

Het onderzoek begint met de bespreking van de online architectuur, waarbij twee verschillende benaderingen worden vergeleken, met nadruk op beveiligingsmaatregelen en hun invloed op de eerlijkheid van het spel.

Daarnaast worden de verschillende netwerkcommunicatiemethoden geanalyseerd, waarbij de voor- en nadelen van elke oplossing worden besproken in de context van het project. Hier wordt er gekozen om ENet te gebruiken.

Vervolgens komt er een bespreking van de software architectuur. De structuur van de software, samen met een uitleg van ieder onderdeel, wordt rijkelijk weergegeven.

Ook worden verschillende game engines geëvalueerd, waarbij Unity wordt geselecteerd vanwege zijn robuuste ondersteuning voor online spellen en uitbundige documentatie.

De spelregels en kaartontwerpen worden uitgewerkt en de implementatie van de kaarten in de software wordt besproken.

Hierna wordt er een toelichting gegeven rond documenten die vaak verschijnen binnen het ontwikkelingsproces van een videospel, maar voor het project achterwege gelaten werden.

Ten slotte volgt een bespreking van hoe de kaarten geïmplementeerd worden en wat ze allemaal inhouden. Ook wordt het verloop van het spel hierin aangekaart samen met het opstellen van je spelkaarten.

Dit onderzoek heeft geleid tot de ontwikkeling van een technisch robuust kaartspel dat werkt met een client-to-server architectuur, gebruikmakend van Unity en ENet. De spelregels zijn uniek voor dit project met een groot aantal uitgewerkte kaarten. De implementatie van de kaarten werd op een modulair en schaalbaar manier gedaan.

Abstract English

The goal of this research is the development of a digital card game that can be played online. Players get to build their own decks and battle against fellow players, with the winner earning new cards.

The research starts with a review of the online architecture used within the project. Two different approaches are compared, with a focus on the security measures and their influence on the fairness of the game.

Following that, there is an analysis of several network communication methods, where every advantage and disadvantage of the solution gets discussed. Ultimately, the choice to use ENet is made.

Next is a discussion of software architecture. The structure of the software, coupled with an explanation of each component, is examined in depth.

Then there is an evaluation of various game engines, with Unity being selected due to its robust support of online games and ample documentation.

Afterwards the game rules together with the card designs get elaborated, with the implementation of the cards within the software being explained.

Then a brief elaboration is given on game design documents, typically used within the development process of a game, that were left out of the project due to being unnecessary for the project.

Lastly, there is a review of how the cards are implemented and what they all contain. The flow of the game also gets depicted together the creation of decks.

In conclusion, this research has led to the development of a technically robust card game that works using client-to-server architecture, using Unity and ENet. The game rules have been designed for this game specifically, with a myriad of unique cards. The implementation of the cards was also done in a scalable and modular way.

Inhoudsopgave

Woord vooraf	i
Abstract Nederlands	ii
Abstract English	iii
Inhoudsopgave	1
Woordenlijst	5
Lijst van afkortingen	6
Lijsten van figuren en tabellen.....	7
Lijst van figuren	7
Lijst van tabellen.....	7
Inleiding	8
1 Situering.....	9
1.1 Wat is een CCG?	9
1.2 Strategie binnen CCG en het belang van kaarteffecten	9
1.3 Verdienmodel van CCG's.....	9
2 Planning	11
2.1 Onderzoek	11
2.2 Minimum Viable Product	11
2.3 Uitbreiding	11
2.4 Finalisatie	11
3 Online Architecturen.....	12
3.1 Peer-to-Peer	12
3.1.1 Voordelen Peer-to-Peer	12
3.1.2 Nadelen Peer-to-Peer	12
3.2 Client-Server Architectuur	13
3.2.1 Voordelen van Client Server	14
3.2.2 Nadeel van Client Server	14
3.3 Hybride Peer-to-Peer en Client-Server.....	14
3.4 Game Discovery	15
3.5 Conclusie	15
4 Netwerkcommunicatie tussen componenten	16
4.1 Eventbusses	16
4.1.1 Event-driven architecture	16

4.1.2	Publish/Subscribe model.....	16
4.1.3	Voorbeelden van eventbusses	17
4.1.4	Nadelen van eventbusses	17
4.2	REST API's.....	18
4.2.1	REST API's en games	18
4.2.2	Polling	19
4.3	Game engine networking	19
4.3.1	Unity Netcode	19
4.3.2	Unreal Engine Networking	19
4.4	Custom protocol.....	20
4.4.1	ENet.....	20
4.5	Conclusie	21
5	Software Architectuur	22
5.1	De kern van software	22
5.2	Gelaagde Architectuur	22
5.2.1	Horizontale lagen	22
5.2.2	Verticale lagen	23
5.3	Dependency Chain	23
5.3.1	Ports and Adapters Interfaces.....	24
5.3.2	Defining dependencies in services.....	24
5.3.3	Infrastructure components	24
5.4	Instantiatie	25
5.5	Use cases	25
5.6	Conclusie	25
6	Game engines.....	26
6.1	Unity	26
6.1.1	Functionaliteit.....	26
6.1.2	Beoordeling.....	26
6.2	Godot.....	26
6.2.1	Functionaliteit.....	27
6.2.2	Netwerkfunctionaliteit	27
6.2.3	Beoordeling.....	27
6.3	LibGDX.....	27
6.3.1	Functionaliteit.....	27
6.3.2	Netwerkfunctionaliteit	27
6.3.3	Beoordeling.....	27

6.4	Clickteam Fusion	28
6.4.1	Functionaliteit.....	28
6.4.2	Netwerkfunctionaliteit.....	28
6.4.3	Beoordeling.....	28
6.5	Photon Engine	28
6.6	Unreal Engine	28
6.6.1	Functionaliteit.....	28
6.6.2	Beoordeling.....	29
6.7	Conclusie	29
7	Regels voor het kaartspel.....	30
7.1	Ontwikkeling van de spelregels.....	30
7.2	Ontwikkeling van de kaarten	30
7.3	Regels	30
7.3.1	Spelopzet.....	30
7.3.2	Kaarttypes en eigenschappen.....	31
7.3.3	Klassen	31
7.3.4	Start van het spel en wincondities	31
7.3.5	Het speelveld	31
7.3.6	Beurtverloop	32
7.3.7	Aanvallen	32
7.4	Regels uit andere games	32
7.4.1	HearthStone.....	32
7.4.2	Yu-Gi-Oh!	33
8	Game design workflow	34
8.1	Art bible	34
8.2	Game design document.....	34
9	Implementatie van kaarten en effecten.....	36
9.1	Structuur en inhoud van kaarten	36
9.1.1	Opslag van kaartgegevens	37
9.1.2	Heroes	38
9.2	De Game-klasse als kerncomponent	38
9.2.1	Gebruik van events	38
9.2.2	Effectensysteem	39
9.2.3	Architecturale keuzes rond de Game-klasse	40
9.3	Users & Decks	40
9.4	Conclusie	40

10	Conclusie	41
11	Referenties.....	43
12	Bijlage	48
12.1	Kaarten	48
12.1.1	Neutral minions	48
12.1.2	Neutral spells:	52
12.1.3	Class specific.....	53
12.1.4	Rarity	59

Woordenlijst

AAA-studio's	Game studio's die games maken in opdracht van een middelgrote of grote uitgever.
AAA-games	Games die uitgegeven worden door een middelgrote of grote uitgever.
Bit-packing	Optimalisatie van data door enkel de minimum nodige bits te gebruiken.
Data Transfer Object	Object gemaakt om data door te geven naar processen. Het bevat geen logica.
Debugging	Fouten in computerprogramma's (software of hardware) detecteren en verbeteren zodat alles naar behoren werkt.
Dependency injection	Een design pattern waarmee men de hardgecodeerde afhankelijkheden kan verwijderen en de applicatie beter onderhoudbaar en testbaar kan maken.
Design pattern	Typische oplossingen voor veelvoorkomende problemen bij het ontwerpen van software.
Domain	Het specifieke onderwerp of het kennisgebied waarop een softwareapplicatie zich richt, inclusief de bijbehorende unieke terminologie, regels,...
Hearthstone	Digitaal kaartspel ontwikkeld door Blizzard.
Interfaces	Afspraken dat een klasse moet volgen. Laat toe om de klassen te identificeren via de interface.
Klasse	Verzameling van data en logica dat een geheel vormt.
Master server	Server dat alle actieve spellen bijhoudt. Wordt gebruikt door een speler om een spel te vinden dat hij kan bijwonen.
Minimum viable product	De eerste versie van een product dat genoeg functionaliteiten bevat om bruikbaar te zijn.
NAT-traversal	Techniek gebruikt om connectie vast te stellen tussen computers die achter NAT zitten.
Netcode	Code geschreven voor netwerkcommunicatie.

Overhead	De extra gegevens en middelen die nodig zijn om het communicatieproces in een netwerk te beheren.
Polling	Het herhaald sturen van aanvragen voor data aan een server om te controleren of een verandering gebeurd is.
Prototype	Model gemaakt om een concept te testen.
Request	Aanvraag gemaakt over het netwerk om bepaalde acties uit te voeren of data te verkrijgen.
Single point of failure	Een kritisch stuk binnen een systeem dat alles kapot maakt als er daar iets verkeerd loopt.
Single source of truth	Enkele punt dat alle waarheid bevat. Wordt gebruikt om inconsistenties te vermijden.
Stateless	Wanneer een object niet zijn eigen toestand bijhoudt.
Tightly coupled	Wanneer componenten hevig afhankelijk zijn aan elkaar.
Trading card game	Kaartspel waar spelers kaarten verzamelen en deze kunnen inzetten tegen elkaar om te strijden
Use case	Manier waarop een product of project gebruikt kan worden.
Yu-Gi-Oh!	Japans kaartspel.

Lijst van afkortingen

- API: Application Programming Interface
- CCG: Collectible Card Game
- CS: Client-Server
- DIP: Dependency Inversion Principle
- DSL: Domain Specific Language
- EDA: Event-Driven Architecture
- GDD: Game Design Document
- MVP: Minimum Viable Product
- NAT: Network Address Translation
- P2P: Peer-to-Peer
- REST API: Representational State Transfer Application Programming Interface
- UI: User Interface
- UML: Unified Modeling Language
- UX: User Experience

Lijsten van figuren en tabellen

Lijst van figuren

Figuur 3.1 Peer-to-Peer-architectuur	12
Figuur 3.2 Client-Server-architectuur	13
Figuur 5.1 Lagen software architectuur	23
Figuur 9.1 Structuur van kaarten	36

Lijst van tabellen

Tabel 2.1 Planning	11
--------------------------	----

Inleiding

Dit bachelor project richt zich op de ontwikkeling van Ironclad Arena, een multiplayer digital collectible card game (CCG) geïnspireerd door spellen zoals Hearthstone en Yu-Gi-Oh!. Spelers kunnen kaarten verzamelen, een deck samenstellen en 1-tegen-1 online wedstrijden spelen. Het doel is het realiseren van een speelbaar prototype dat de kernmechanieken van een digitaal kaartspel duidelijk demonstreert. De keuze van deze inspiratiebronnen en de specifieke spelmechanieken en features die hieruit werden overgenomen, worden verder in dit document toegelicht.

De studenten onderzochten in deze thesis elk onderwerp die een grote rol speelt bij het ontwikkelen van een digitaal kaartspel en is in deze tekst gepresenteerd, waar een gezonde hoeveelheid generatieve artificiële intelligentie geraadpleegd werd.

Het project omvat verschillende software technische aspecten, waaronder game-logica, state management, client-servercommunicatie, UI/UX-ontwikkeling en het toepassen van professionele software engineeringpraktijken zoals architectuurontwerp, design patterns en teamwork. Ook zijn onderzoeks- en beslissingsmomenten essentieel, zoals de keuze van de technologieën, de netwerkachitectuur en manieren om de integriteit van de spelstatus te bewaken.

De teamleden beschikken over uiteenlopende voorkennis, waardoor niet iedereen even vertrouwd was met game development of het implementeren van netcode. Dankzij onderlinge ondersteuning, continue communicatie en het raadplegen van relevante bronnen kan ieder groepslid een betekenisvolle technische bijdrage leveren en groeit de algemene kennis rond game-ontwikkeling en software engineering.

Het eindresultaat is een werkend prototype met deckselectie, speelbare kaarten, duidelijke spelregels en basis wincondities. Daarnaast beschikt de game over een functionerende online multiplayer, een grafische interface en een beperkte maar representatieve set kaarten. Dit prototype vormt een solide basis voor verdere uitbreidingen en illustreert de toepassing van professionele ontwikkelpraktijken binnen een concreet softwareproject.

1 Situering

Om het ontwerp en de implementatie van een digitaal collectible card game (CCG) te kunnen analyseren, is het noodzakelijk om het genre en zijn onderliggende principes te situeren. Dit hoofdstuk bespreekt wat een CCG is, welke rol strategie en kaarteffecten spelen binnen dit type spel, hoe het verdienmodel van CCG's doorgaans is opgebouwd. Deze context vormt de theoretische basis voor de ontwerpkeuzes die verder in deze bachelorproef worden gemaakt.

1.1 Wat is een CCG?

Collectible card games vinden hun oorsprong in verzamelkaarten, waarbij spelers kaarten verzamelen die bestaan in verschillende zeldzaamheidsklassen. In tegenstelling tot traditionele verzamelkaarten zijn CCG's expliciet ontworpen om met deze kaarten te spelen. [1] Elke kaart bevat spelinformatie die bepaalt hoe zij binnen het spel kan worden ingezet, zoals kosten, voorwaarden en effecten. Hierdoor fungeren kaarten niet enkel als verzamelobjecten, maar ook als actieve spelcomponenten. [2]

Hoewel er meerdere definities van collectible card games bestaan, vertonen zij gemeenschappelijke kernmerken. Een CCG is een spelvorm waarin spelers gebruikmaken van verzamelbare kaarten die regels en effecten bevatten en strategisch worden ingezet om het spelverloop te beïnvloeden. Zowel je fysieke als digitale varianten delen deze eigenschappen. In digitale CCG's verzamelen spelers virtuele objecten die persistent blijven over meerdere speelsessies en een blijvende impact hebben op toekomstige spellen. [3]

1.2 Strategie binnen CCG en het belang van kaarteffecten

Strategie vormt een essentieel onderdeel van collectible card games en is nauw verbonden met de effecten die kaarten kunnen uitoefenen. Veel CCG's maken gebruik van terugkerende spelmechanieken, zoals kaarten die grondstoffen voorstellen, eenheden die kunnen worden ingezet in gevechten en kaarten die andere elementen versterken of wijzigen. [4] Deze basismechaniek zorgen voor een herkenbare structuur binnen het genre.

Wat CCG's onderscheidt van andere spelvormen is de manier waarop individuele kaarten uitzonderingen creëren op de algemene spelregels. [5] Elke kaart kan specifieke effecten bevatten die bestaande regels uitbreiden of doorbreken, wat leidt tot een grote variatie aan mogelijke speltoestanden. Hierdoor ontstaat een spelervaring waarin deckbuilding, strategische planning en situationele besluitvorming samenkomen. Dit maakt het ontwerpen en balanceren van CCG's bijzonder complex, zeker wanneer het verzamelen van kaarten en hun beschikbaarheid een directe invloed hebben op de competitieve verhoudingen tussen spelers. [3]

1.3 Verdienmodel van CCG's

Het verdienmodel van CCG's is sterk verbonden met het verzamelaspect van het genre. Spelers starten doorgaans met een basisdeck dat hen toelaat het spel te leren en te spelen.

Naarmate zij meer ervaring opdoen, kunnen extra kaarten worden verkregen, vaak via de aankoop van boosters met een willekeurige inhoud. Deze kaarten worden vervolgens geïntegreerd in bestaande decks om nieuwe strategieën mogelijk te maken of bestaande decks te versterken. [6]

Dit model wordt zowel bij fysieke als digitale CCG's toegepast en vormt een belangrijk economisch fundament voor het genre. De verkoop van kaarten en uitbreidingen genereert een continue inkomensstroom, maar heeft tegelijkertijd een aanzienlijke invloed op het spelontwerp. Ontwerpers moeten rekening houden met spelbalans, kaartzeldzaamheid en progressie, zodat het spel zowel aantrekkelijk blijft voor spelers als economisch duurzaam is. [7]

2 Planning

De ontwikkeling van deze bachelorproef werd uitgevoerd in teamverband en gestructureerd volgens een gefaseerde planning. Deze aanpak maakte het mogelijk om het project systematisch op te bouwen, van conceptuele voorbereiding tot een afgewerkt en getest eindproduct, deze fases kan je zien in Tabel 2.1.

Tabel 2.1 Planning

	W1	W2	W3	W4	W5	W6	W7	W8
Onderzoek								
MVP								
Uitbreiding								
Finalisatie								

2.1 Onderzoek

In de onderzoeksfase werd de inhoudelijke en technische basis van het project gelegd. Er werd onderzoek uitgevoerd naar geschikte technologieën en bestaande kaartspellen ter inspiratie. Op basis hiervan werden het spelconcept, de spelregels en de kaartstructuur uitgewerkt en iteratief verfijnd, onder meer door het testen van fysieke prototypes. Daarnaast werd de technische haalbaarheid geëvalueerd, met aandacht voor netwerkcommunicatie tussen client en server. Parallel werd gestart met het schrijven van de bachelorproef en het opstellen van ondersteunende modellen.

2.2 Minimum Viable Product

Tijdens de Minimum Viable Product (MVP) fase lag de focus op het realiseren van een eerste functionele en speelbare versie van het spel. De kernarchitectuur en het domeinmodel werden geïmplementeerd, samen met essentiële spelmechanieken, netwerkfunctionaliteit en een basisgebruikersinterface. Door middel van testen en iteratieve verbeteringen werd een stabiel prototype ontwikkeld dat de kern van het spel correct weerspiegelde.

2.3 Uitbreiding

Na het realiseren van het MVP werd het spel verder uitgebreid in de uitbreidingsfase. In deze fase werden bijkomende functionaliteiten geïmplementeerd, terwijl het MVP verder werd verfijnd. Ook het visuele aspect van het spel werd verder uitgewerkt. Tegelijkertijd werd de bachelorproef inhoudelijk gefinaliseerd door redactionele verbeteringen, het verwerken van feedback en het vervolledigen van de documentatie.

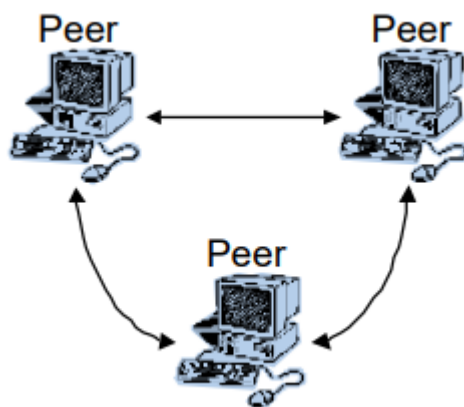
2.4 Finalisatie

De finalisatiefase was gericht op kwaliteitsverbetering en afronding van het project. Hierbij werd extra aandacht besteed aan testen en debugging, evenals aan een gezamenlijke evaluatie van de volledige codebase. Eventuele kleine optimalisaties werden doorgevoerd om te komen tot een stabiel, onderhoudbaar en afgerond eindresultaat.

3 Online Architecturen

Online architecturen beschrijven de manier waarop het netwerkverkeer voor een spel georganiseerd wordt. Aangezien spelers van een digitaal kaartspel zich op verschillende machines bevinden, moet informatie efficiënt en betrouwbaar van de ene naar de andere machine verstuurd kunnen worden. De structuur van deze communicatie hangt af van de gekozen netwerkarchitectuur. In de literatuur worden hoofdzakelijk twee relevante modellen besproken: Peer-to-Peer (P2P) en Client-server (CS). De keuze tussen beide is volgens ETH Zürich [8] (2003) afhankelijk van het type probleem en de vereisten van de game.

3.1 Peer-to-Peer



Figuur 3.1 Peer-to-Peer-architectuur

In een Peer-to-Peer-architectuur zijn alle clients gelijkwaardig. Het netwerk is gedecentraliseerd en iedere client communiceert direct met andere clients zonder tussenkomst van een centrale server zoals je kan zien in figuur 3.1.

3.1.1 Voordelen Peer-to-Peer

3.1.1.1 Geen single point of failure

Wanneer één client uitvalt, kan de rest van het netwerk blijven functioneren. Dit is vooral waardevol in grootschalige netwerken waar centralisatie een risicofactor vormt.

3.1.1.2 Minder serverinfrastructuur nodig

Omdat er geen centrale server wordt gehost, dalen de vereisten op het vlak van onderhoud en hostingkosten. Voor een project zonder eigen serverinfrastructuur is dit aantrekkelijk

3.1.2 Nadelen Peer-to-Peer

Ondanks bovenstaande voordelen brengt het P2P-model significante uitdagingen met zich mee:

3.1.2.1 Beveiligingsproblemen

In een P2P-netwerk bevindt de spel-logica zich op elke client. Dit maakt het systeem kwetsbaar voor valsspelen en hacking, omdat gemodificeerde clients de spelregels kunnen manipuleren. GetGud (2023) [9] benadrukt dat P2P hierdoor bijzonder gevoelig is voor valsspelen. Enkele voorbeelden van waarom client-server vals spelen tegen gaat:

- Hacked clients kunnen meer informatie bekijken dan zichtbaar zou mogen zijn in het spel.
- Een client kan game informatie vervalsen en andere clients overtuigen dat de vervalste informatie correct is, waardoor een speler een oneerlijk voordeel kan krijgen.

In Ironclad Arena zou een speler bijvoorbeeld de levenspunten van de held kunnen vervalsen door absurd hoge waarde toe te kennen, dit zou ervoor zorgen dat een speler langer kan blijven overleven dan toegestaan volgens de regels.

P2P clients kunnen hier niet vlot mee overweg waardoor een systeem met hoge beveiligingsvereisten best geen P2P netwerkarchitectuur gebruikt.

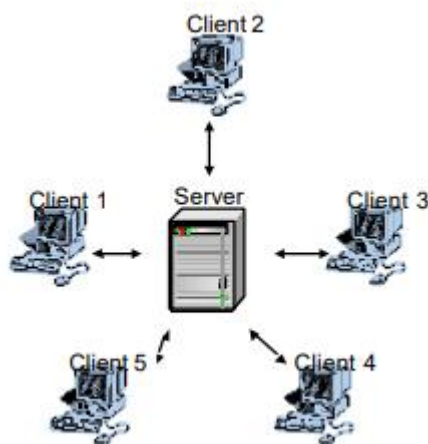
3.1.2.2 Inconsistenties in gamestate

Het synchroniseren van spelers, kaarten en spelgegevens is complex omdat elke client een eigen versie van de gamestate bijhoudt. Het consistent houden van deze staten over meerdere machines is foutgevoelig, vooral wanneer netwerkvertraging of pakketverlies optreedt.

3.1.2.3 Complexiteit van netwerkbeheer

Bij peer-to-peer netwerken komen verschillende complexiteiten naar voren, zoals NAT-traversal, pakketdistributie en het waarborgen van gelijke toegang tussen peers. Deze aspecten vormen echter een technische uitdaging die te gedetailleerd is om hier verder uit te diepen. [10]

3.2 Client-Server Architectuur



Figuur 3.2 Client-Server-architectuur

In een Client–Server-architectuur communiceren alle spelers (clients) met één centrale autoritaire server. De server fungeert als single source of truth, verwerkt alle spelacties en

stuurt de correcte gamestate terug naar de clients. De clients communiceren niet rechtstreeks met elkaar. In figuur 3.2 kan je zien dat elke client slechts één connectie heeft met de centrale server. [8]

3.2.1 Voordelen van Client Server

3.2.1.1 Beperkte eisen aan de client

Client moeten minder rekenkracht en logica bevatten, omdat de server het grootste deel van verwerking uitvoert.

3.2.1.2 Consistente data en eenvoudiger beheer

Doordat één server de autoriteit heeft over de gamestate, blijven inconsistenties minimaal. Datamanagement, synchronisatie en validering worden centraal uitgevoerd.

3.2.1.3 Betere beveiliging

Valsspelen wordt aanzienlijk moeilijker, omdat de server elke actie controleert en kan weigeren. Manipulaties van health, posities of verborgen informatie worden hierdoor onmogelijk [9].

3.2.1.4 Eenvoudiger netwerkmodel

De communicatie verloopt enkel tussen elke client en de server. Dit maakt het netwerk betrouwbaarder en gemakkelijker te implementeren.

3.2.2 Nadeel van Client Server

3.2.2.1 Hogere onderhoudslast

Er moeten twee projecten ontwikkeld en onderhouden worden: de client en de server. Dit brengt extra complexiteit met zich mee.

3.2.2.2 Single point of failure

Wanneer de server uitvalt, kan niemand verder spelen. Hoge beschikbaarheid en monitoring zijn daarom cruciaal.

3.3 Hybride Peer-to-Peer en Client-Server

Sommige games gebruiken een hybride systeem waarbij dynamisch geschakeld wordt tussen P2P en CS, afhankelijk van de nabijheid tot servers of netwerkcondities. Dit wordt onder meer besproken in Gears of War's GDC-presentatie. [11] Deze architectuur is echter complex en vooral relevant voor grootschalige real-time games met hoge performantie-eisen. [12]

Voor ons project is dit scenario niet van toepassing, aangezien de voordelen niet opwegen tegenover de bijkomende complexiteit.

3.4 Game Discovery

Net zoals er tijdens het spelen van games doorgaans een client–servermodel ondersteund wordt, kan je ook het zoeken van andere spelers laten verlopen via het client-servermodel. Dit proces omvat het registreren van beschikbaar gamesessies en het aanbieden van lijstjes aan spelers. [13]

Onderzoek binnen de architecturen van ‘first person shooters’ toont aan dat dit meestal gebeurt via een master server (Swinburne University, 2024) [14], die alle beschikbare games bijhoudt. Dit is een alleenstaande server die clients gebruiken om afzonderlijke game servers te ontdekken. Hierdoor bevordert je de infrastructuur en performantie van een game maar dit vereist echter een extra servercomponent en is voor dit project te zwaar en complex.

Hoewel dergelijke systemen schaalbaar en performant zijn, volstaan voor beperkte toepassing eenvoudigere modellen waarbij één server zowel gameplay als discovery afhandelt.

3.5 Conclusie

Op basis van de analyse van verschillende netwerkkarchitecturen blijkt dat een Peer-to-Peer-oplossing te veel risico's inhoudt op het vlak van beveiliging, synchronisatie en netwerkcomplexiteit. Het Client-Server model biedt daarentegen een robuuste en veilige manier om speldata consistent te houden, valsspelen te voorkomen en de spelervaring betrouwbaar te maken.

Ironclad Arena gebruikt daarom een autoritaire server die verantwoordelijk is voor de volledige spellogica en state-validatie. De clients sturen enkel acties door en visualiseren de informatie die door de server wordt aangeleverd. Dit zorgt voor een duidelijke taakverdeling, betere beveiliging en een stabiele gebruikerservaring.

4 Netwerkcommunicatie tussen componenten

Wanneer verschillende softwarecomponenten, zoals een client en een server in een spelomgeving, met elkaar moeten communiceren, is een geschikt communicatieprotocol essentieel. Ongeacht de gebruikte netwerktopologie wordt deze communicatie doorgaans gerealiseerd via een Application Programming Interface (API). Een API fungeert als een gestandaardiseerde tussenlaag die definieert hoe data kan worden uitgewisseld tussen systemen of applicaties. Volgens Lem Apperson (2023) [15] bieden API's een consistente en gestructureerde manier om functionaliteit toegankelijk te maken voor andere software, zowel binnen traditionele applicaties als in gameontwikkeling.

Er bestaan verschillende benaderingen om netwerkcommunicatie te implementeren, maar een vaak onderzochte optie is het gebruik van een eventbus.

4.1 Eventbusses

Een eventbus is een mechanisme dat toelaat om gebeurtenissen (events) asynchroon over een systeem of netwerk te versturen. In essentie fungeert de eventbus als een centraal communicatiekanaal waarop berichten gepubliceerd kunnen worden zonder dat de verzender moet weten wie de ontvanger is. Dit maakt eventbusses een mogelijke manier om losse, schaalbare en minder strak gekoppelde systemen te realiseren. [16]

4.1.1 Event-driven architecture

Eventbusses worden vaak gebruikt binnen een event-driven architecture (EDA). Het is een systeem waarin componenten reageren op gebeurtenissen die worden gepubliceerd naar een centraal punt. EDA werkt volgens het publish/subscribe-model, waarbij communicatie niet rechtstreeks tussen twee componenten gebeurt, maar via de eventbus.

4.1.2 Publish/Subscribe model

4.1.2.1 Publishers

Publishers zijn componenten die gebeurtenissen genereren en publiceren naar de eventbus. Een publisher detecteert een wijziging of actie binnen het systeem en zet deze om in een event. Hierbij is de publisher volledig ontkoppeld van de ontvangers: hij hoeft niet te weten welke componenten het event zullen ontvangen of verwerken. Dit zorgt ervoor dat publishers eenvoudig blijven en niet afhankelijk zijn van andere onderdelen van het systeem.

4.1.2.2 Subscribers

Subscribers zijn componenten die zich abonneren op één of meerdere soorten events. Wanneer een relevant event wordt gepubliceerd, ontvangt de subscriber dit automatisch en voert hij de bijhorende logica uit. Subscribers kunnen dynamisch worden toegevoegd of verwijderd zonder dat dit invloed heeft op de publishers.

Door deze vorm van ont koppeling kunnen systemen eenvoudiger worden uitgebreid en aangepast. Nieuwe functionaliteiten kunnen worden toegevoegd door nieuwe subscribers te introduceren, zonder dat bestaande componenten gewijzigd moeten worden. Dit maakt het

publish/subscribe-model bijzonder geschikt voor modulaire en schaalbare softwarearchitecturen.

4.1.3 Voorbeelden van eventbusses

Er bestaan implementaties van eventbusses in vrijwel elke grote programmeertaal:

- Vert.x Eventbus (Java/Kotlin)
Vert.x bevat een ingebouwde eventbus die communicatie tussen componenten en tussen meerdere instanties ondersteunt. Ze wordt vaak gebruikt voor gedistribueerde systemen en webapplicaties. [17]
- Guava EventBus (Java)
Google biedt met Guava een EventBus-package dat een eenvoudige manier biedt om publish/subscribe-interacties binnen één applicatie te implementeren. [18]
- ws (JavaScript)
In JavaScript bestaat de ws-package, die WebSocket-functionaliteit ondersteunt. Hoewel dit niet strikt een eventbus is, wordt het in veel projecten gebruikt om event-achtige communicatie te implementeren tussen clients en servers. [19]
- Unity Eventbus
Binnen Unity bestaan community-gedreven implementaties van eventbusses die specifiek geïntegreerd kunnen worden in Unity-projecten. Ze vergemakkelijken de interne communicatie tussen gameobjecten en systemen. [20]

Hoewel er dus een breed aanbod aan eventbus oplossingen bestaat, is interoperabiliteit vaak een uitdaging. Veel eventbusses zijn taal- of frameworkspecifiek, waardoor het niet vanzelfsprekend is dat een Java-eventbus probleemloos communiceert met een JavaScript- of C#-eventbus. Dit kan problematisch zijn bij projecten waar client en server in verschillende talen ontwikkeld worden.

4.1.4 Nadelen van eventbusses

Hoewel eventbusses aantrekkelijk zijn door hun losgekoppelde ontwerp, zijn er ook belangrijke nadelen, zeker in de context van game development.

4.1.4.1 Debugging Complexiteit

Op Unity-discussiefora wordt aangegeven dat eventbusses het debuggen bemoeilijken. Omdat publishers en subscribers niet rechtstreeks aan elkaar gekoppeld zijn, is het minder duidelijk waar een bepaalde gebeurtenis vandaan komt of waarom een bepaald event werd geactiveerd. [21]

4.1.4.2 Performantie

Veel games zijn van nature ‘tightly coupled’ omdat gameplay-systemen sterk afhankelijk zijn van snelle en voorspelbare communicatie. Eventbusses introduceren een extra abstractie laag, wat kan leiden tot vertragingen of meer overhead, zeker bij een hoge frequentie van events.

4.1.4.3 Beveiligingsrisico's

In netwerkcontexten kan een eventbus beveiligingsproblemen opleveren. Volgens bijdragen op GameDev.net zijn eventbus-architecturen gevoeliger voor ongewenste of schadelijke input, omdat events vaak breed in het systeem worden verspreid. Zonder strikte validatie kunnen kwaadaardige events een groot deel van het systeem beïnvloeden. [22]

4.1.4.4 Interoperabiliteitsproblemen

Omdat veel eventbusses framework- of taalgebonden zijn, is het niet vanzelfsprekend dat verschillende systemen (bijvoorbeeld een Unity-client en een Java-server) naadloos met elkaar communiceren. Dit verhoogt het risico dat ontwikkelaars pas laat in het proces ontdekken dat hun gekozen eventbus technologie niet geschikt is voor cross-platform communicatie.

4.2 REST API's

Een veelgebruikte methode voor communicatie tussen een client en een server is het gebruik van een REST API (Representational State Transfer). Volgens IBM (2023) [23] is een REST API een gestandaardiseerde manier om objecten over het netwerk te versturen, waarbij deze objecten de toestand (state) van de server representeren. REST koppelt de client los van de server door gebruik te maken van uniforme interfaces en stateless communication.

In een REST-architectuur bevat elk request alle noodzakelijke informatie om deze volledig te kunnen verwerken. De server bewaart geen context tussen verschillende requests, wat de schaalbaarheid verhoogt maar ook meer overhead introduceert. In de context van games kan deze statelessness een nadelig effect hebben: elke actie moet opnieuw volledige contextinformatie versturen, wat inefficiënt kan worden bij frequente updates of snelle gameplay.

Bovendien werkt REST volledig volgens het request–response principe, waardoor de server niet zelfstandig informatie kan opsturen naar de client. Dit maakt het ongeschikt voor scenario's waarin real-time updates cruciaal zijn, zoals in snelle multiplayer games.

4.2.1 REST API's en games

Binnen gameontwikkeling is een REST API voornamelijk geschikt voor relatief trage of turn-based games. GameDev StackExchange (2020) [24] geeft aan dat REST bruikbaar is wanneer lage updatefrequentie, beperkte asynchroniteit en traag spelverloop aanvaardbaar zijn. Hoewel ons project technisch binnen een turn-based categorie valt, is REST niet ideaal omdat wij een efficiëntere, meer gamegerichte aanpak willen verkennen.

4.2.2 Polling

Om het gebrek aan een manier om de server te laten initialiseren in REST te mitigeren, kan polling worden gebruikt. Hierbij stuurt de client op vaste intervallen requests naar de server om recente updates op te vragen. Volgens Sankalpa (2021) [25] kan polling een oplossing vormen om alsnog tijdsgevoelige informatie op te halen, maar het is minder performant omdat voortdurend communicatie plaatsvindt, ongeacht of er nieuwe data is.

Hoewel polling een gedeeltelijke oplossing biedt voor het gebrek aan real-time server initiatie, blijft het suboptimaal voor moderne multiplayer games.

4.3 Game engine networking

Bepaalde game-engines beschikken over ingebouwde netwerktechnologie, specifiek ontwikkeld voor multiplayer games.

4.3.1 Unity Netcode

Unity biedt een eigen netwerkframework, Unity Netcode [26], waarmee zowel Peer-to-Peer als Client-Servermodellen ondersteund worden. Deze oplossing bevat kant-en-klare functionaliteiten, synchronisatiemanagement en serverintegraties, wat het aantrekkelijk maakt voor veel gameprojecten.

4.3.1.1 Server Integreren in Unity

Unity laat ook toe om de server zelf in Unity te implementeren. Dit heeft als voordeel dat een client tijdelijk als server kan fungeren, waardoor het spel altijd speelbaar blijft. Daarnaast zijn alle Unity-functionaliteiten beschikbaar aan de kant van de server. [27]

De Unity-documentatie (2024) [28] benadrukt dat netcode nauw aansluit bij Unity's objectmodel en workflow. Hoewel dit voordelen biedt voor projecten die volledig in Unity draaien, is het minder geschikt wanneer men een strikte scheiding tussen client en server wil behouden. Een server geschreven in Unity bevat bovendien aanzienlijke overhead omdat de volledige engine geladen moet worden, zelfs wanneer enkel netwerkfunctionaliteit nodig is. Daarom is een externe serverarchitectuur in dit project wenselijker en efficiënter.

Toch is dit voor dit project niet ideaal:

- De Unity server brengt onnodige overhead mee;
- Alle logica blijft sterk verweven met de Unity-structuur;
- Een externe serverarchitectuur biedt een veel heldere scheidingslijn tussen client en server.

Aangezien Ironclad Arena weinig voordeel haalt uit Unity's ingebouwde serverfunctionaliteiten, wordt deze optie verworpen ten gunste van een lichtere, custom oplossing.

4.3.2 Unreal Engine Networking

Ook Unreal Engine biedt een uitgebreid en performant netwerk-ecosysteem dat ontworpen is voor real-time multiplayeromgevingen. [29] Ondanks de kracht van deze systemen geldt

dezelfde overweging: wanneer men een engine-agnostische serverarchitectuur wil of een lichte server zonder engine-overhead, is een custom of extern systeem meer geschikt.

Hoewel engines netwerkfunctionaliteit bieden, betekent het ontbreken daarvan in een engine niet dat geen netwerkverbinding mogelijk is. In dat geval moet de netwerk laag zelf ontwikkeld worden, een proces dat intensief kan zijn maar volledige controle biedt over de serverlogica.

4.4 Custom protocol

Een alternatieve en flexibele benadering is het ontwikkelen van een custom netwerkprotocol. Dit sluit het best aan bij projecten waarin volledige controle, minimalistische overhead en cross-platform compatibiliteit belangrijk zijn. [30]

Vaak worden custom protocollen opgebouwd rond serialisatie van objecten naar compacte binaire structuren, die vervolgens over het netwerk worden verstuurd. Dit vereist doorgaans een gedeelde codebase voor Data Transfer Objects (DTO's), waardoor server en client exact dezelfde gegevensstructuren interpreteren.

Aan de ene kant vraagt dit extra ontwikkeltijd, maar aan de andere kant biedt een eigen protocol maximale controle over:

- Performantie
- Datacompressie
- Foutafhandeling
- Netwerkverkeerbeheer

Daarnaast laat dit toe om technieken zoals compressie toe te passen wanneer dit noodzakelijk wordt. Door data te optimaliseren (bijvoorbeeld via bit-packing of compressie-algoritmen), kan de hoeveelheid verstuurd informatie drastisch worden verminderd.

4.4.1 ENet

ENet versimpelt het opzetten van een client-server architectuur omdat het kan gebruikt worden voor de client en de server. Hierdoor blijft de netcode uniform en eenvoudig. [31]

ENet werkt met meerdere kanalen waarover packets verstuurd kunnen worden. Ieder kanaal kan dus een unieke game bevatten, dit vormt een goede barrière tussen de games en hun state. De spelers binnen in het spel worden dan weer onderscheiden via een ID.

Via flags kan er gekozen worden hoe betrouwbaar een packet verstuurd wordt. Aangezien het aantal packets tot een minimum gehouden zal worden, zullen enkel noodzakelijke packets verstuurd worden waardoor ze altijd betrouwbaar aankomen.

De data binnen een packet is een verzameling van integers. Via afgesproken regels wordt er uniformiteit gegeven aan de verzameling en worden packets onderscheiden van elkaar. Dit zorgt ervoor dat communicatie tussen client en server vlot verloopt.

Events maken het makkelijk om te reageren op de communicatie tussen de client en server. Er wordt een onderscheid gemaakt tussen deze events: connecteren, ontkoppelen, geen communicatie gedurende een bepaalde tijdspanne en ontvangen van een packet.

Dit versimpelt het bijhouden van connecties. Enkel bij het connecteren en ontkoppelen moet een speler toegevoegd en verwijderd worden.

De server draait op een simpel IPv4 adres en poort. De poort en adres kunnen gekozen worden bij het opstarten van de server.

4.5 Conclusie

Binnen het domein van game networking bestaan er diverse benaderingen, variërend van REST API's en polling tot engine-geïntegreerde oplossingen en volledig custom protocollen. REST en polling zijn werkbaar voor turn-based systemen maar brengen te veel overhead en beperkingen met zich mee voor een moderne multiplayerarchitectuur. Engine-gebaseerde oplossingen zoals Unity Netcode of Unreal Networking zijn krachtig, maar minder geschikt wanneer een lichte, engine-onafhankelijke server gewenst is.

Voor Ironclad Arena biedt een custom protocol, ondersteund door ENet, de beste balans tussen prestatie, flexibiliteit en controle. Het maakt een duidelijke scheiding mogelijk tussen client en server en vermijdt de beperkingen van alternatieve frameworks, terwijl het een efficiënte communicatie laag biedt voor een turn-based gameomgeving.

5 Software Architectuur

Software Architectuur vormt een essentieel fundament voor de ontwikkeling van elk stuk software. Een duidelijke structuur voorkomt coupling tussen componenten en maakt het eenvoudiger om functionaliteit terug te vinden, aan te passen en uit te breiden. In gameontwikkeling is dit een moeilijke evenwichtsoefening: code moet leesbaar en onderhoudbaar blijven. Maar in de context van een spel moet code ook performant zijn en moet je snel kunnen prototypen. Deze doelstellingen vallen zelden volledig te combineren, waardoor voortdurend moet worden afgewogen tussen snelheid en kwaliteit. [32]

Prototyping is noodzakelijk om ideeën snel te testen, maar de bijhorende code is tijdelijk en moet bewust worden weggegooid zodra ze haar doel heeft bereikt. Code die op lange termijn gebruikt wordt, vereist meer zorg en structuur. [33] Om dit te onderhouden, worden binnen het project regelmatig code reviews uitgevoerd. Deze zorgen voor een consistente en onderhoudbare broncode en maken toekomstige uitbreidingen efficiënter.

5.1 De kern van software

Binnen een softwaresysteem vormt de domeinlaag het centrale onderdeel, omdat deze de kernlogica van het domein vastlegt. Voor een online kaartspel omvat dit de formele spelregels, de interacties tussen kaarten en spelers, en de onderliggende spelmechanismen. In lijn met de principes van Domain-Driven Design (DDD) [34] wordt deze laag ontworpen als een directe, begrijpelijke representatie van het domein, zodat de code een nauwkeurige vertaling vormt van de spelregels. Door gebruik te maken van een gedeelde, domeinspecifieke taal kunnen zowel technische als niet-technische betrokkenen de implementatie en het ontwerp beter begrijpen.

DDD biedt bovendien economische en structurele voordelen. Door complexe zakelijke logica expliciet te modelleren en te isoleren in het domein, wordt het eenvoudiger om gedrag terug te vinden, aan te passen of uit te breiden naarmate de applicatie groeit. Dit leidt tot een hogere flexibiliteit en beperkt de kosten van onderhoud en verdere ontwikkeling.

5.2 Gelaagde Architectuur

Een veelgebruikte benadering binnen softwareontwikkeling is het opdelen van de applicatie in afzonderlijke lagen. Deze lagen worden onderling verbonden via expliciete afhankelijkheden. De indeling gebeurt idealiter op basis van hun “reden tot veranderen”, in overeenstemming met het Single Responsibility Principle (SRP). [35] Door elke laag verantwoordelijk te maken voor één duidelijk afgebakend aspect van het systeem, wordt de kans op frequente herwerking verminderd en blijft de codebase beter onderhoudbaar. [36]

5.2.1 Horizontale lagen

Eén manier om software op te splitsen is afhankelijk van hoe sterk je vasthangt aan externe technologieën. Hieronder vallen twee manieren om de code op te delen. Je kan de domainlaag van onder plaatsen, onder de lagen die steeds dichter aansluiten bij externe systemen zoals gebruikersinterfaces en databanken. Deze opbouw is aantrekkelijk omdat de

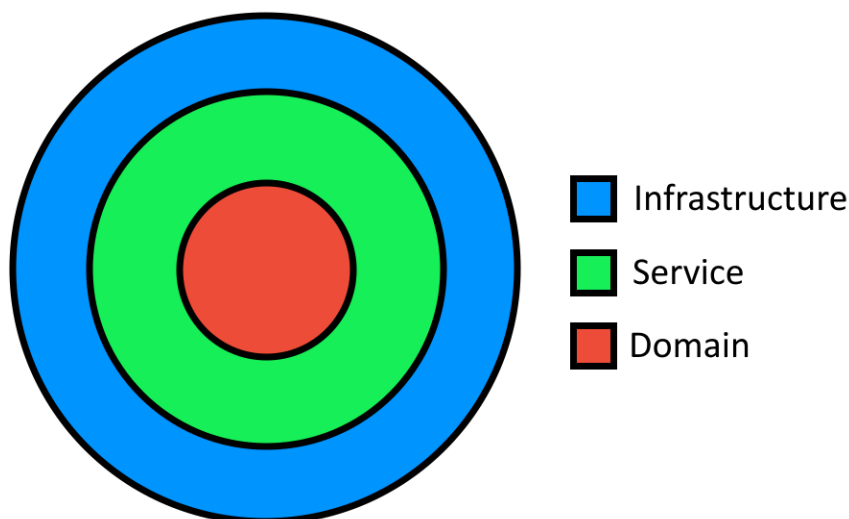
domeinlogica volledig onafhankelijk blijft van technische implementaties of externe componenten. [37]

Een alternatieve horizontale structuur bestaat uit een scheiding waar je de domain-laag in het midden plaatst van andere systemen, hier kan de interactielogica dicht bij de gebruiker staan, terwijl de dataverwerking meer intern en minder zichtbaar is. De buitenste lagen zijn hier het dichtste bij externe technologieën. Dit wordt ook wel een technische indeling genoemd.

5.2.2 Verticale lagen

Een verticale indeling groepeert componenten op basis van specifieke use cases. Deze structuur helpt te voorkomen dat logica onbedoeld wordt gedeeld tussen verschillende contexten, waardoor afhankelijkheden binnen het domein vermeden worden. Er ligt echter geen restrictie op de koppeling van externe componenten en het domein, wat de abstractie en uitwisselbaarheid van technische elementen kan verkleinen. [37]

5.3 Dependency Chain



Figuur 5.1 Lagen software architectuur

Om horizontale lagen binnen software samen te brengen kunnen we afhankelijkheden leggen tussen de lagen. Deze afhankelijkheden leggen we hiërarchisch, de bovenste laag, die we gebruiken om alles samen te koppelen, mag afhankelijk zijn van elke andere laag. De onderste laag, wat onze domain logica bevat, mag juist geen afhankelijkheden hebben. [32] Figuur 5.1 schetst de lagen die aanwezig zijn binnen Ironclad Arena en toont dat Domain volledig gescheiden is van infrastructuur.

Het is belangrijk om de domeinlaag onafhankelijk te maken van elk ander component omdat het de regels van een echt kaartspel zou moeten voorstellen als code. Als je het spel zou spelen in het echt heb je helemaal geen netwerk of databank nodig, als gevolg mogen we geen infrastructuur verwerken in het domein. [34]

5.3.1 Ports and Adapters Interfaces

In de binnenste lagen wordt eerst vastgelegd welke functionaliteit noodzakelijk is om de use cases te ondersteunen. Deze vereisten worden geformuleerd als required interfaces. De infrastructuurlaag levert vervolgens de bijhorende provided interfaces aan en past zich daarmee aan de noden van het domein aan, zonder dit domein afhankelijk te maken van technische details.

Deze structuur sluit aan bij het onderscheid tussen driving en driven adapters. Driving adapters bieden input of gebeurtenissen aan het domein aan, terwijl driven adapters door het domein worden aangestuurd om resultaten naar buiten te brengen. In een online trading card game geldt dit zowel voor netwerkcommunicatie als voor de gebruikersinterface, die elk in bepaalde situaties als driving of driven adapter functioneren.

5.3.2 Defining dependencies in services

Het Dependency Inversion Principle (DIP) [32] speelt een cruciale rol bij het afbakenen van verantwoordelijkheden binnen de domeinlaag. Dit principe stelt dat de onderste componenten afhankelijk moeten zijn van abstracties, niet van concrete implementaties. Door enkel afhankelijkheden te leggen op wat nodig is en niet hoe het gebeurt, wordt het mogelijk om eerst vast te leggen welke functionaliteit conceptueel vereist is, waarna de concrete infrastructuurimplementaties deze verwachtingen kunnen invullen.

Deze werkwijze maakt de domeinlaag onafhankelijk van technologische keuzes en bevordert testbaarheid, uitbreidbaarheid en vervangbaarheid van infrastructuurcomponenten. Bovendien wordt hierdoor voorkomen dat domeinservices rechtstreeks gekoppeld worden aan specifieke frameworks, databases of netwerkprotocollen.

5.3.3 Infrastructure components

Infrastructuurcomponenten kunnen in brede zin worden opgevat als adapters [38]: zij leveren de concrete implementaties die voldoen aan de door het domein gedefinieerde interfaces. Voorbeelden hiervan zijn netwerklagen, databanken of externe frameworks. Door deze componenten strikt als adapters te structureren, kunnen technologische keuzes worden aangepast zonder invloed op de domeinlogica.

Extra functionaliteit, zoals integratie met anti-cheatoplossingen, betalingssystemen of andere externe diensten, kan op dezelfde manier binnen de infrastructuurlaag worden ondergebracht. Dit maakt het ook mogelijk externe diensten te vervangen of te evolueren, bijvoorbeeld het wisselen tussen databanken, zonder architecturale herstructurering op domeinniveau. De flexibiliteit die hierdoor wordt bereikt, ondersteunt zowel de korte- als langetermijnontwikkeling van de applicatie.

5.4 Instantiatie

In de gehanteerde architectuur wordt instantiatie opgevat als het moment waarop concrete implementaties van abstracte componenten worden aangemaakt. Hoewel dit een noodzakelijk onderdeel is van het systeem, wordt het beschouwd als een detail dat idealiter niet willekeurig binnen de applicatie verspreid wordt. Om deze reden wordt een afzonderlijke laag geïntroduceerd, deze noemt “main” zoals de startfuncties in vele programmeertalen. Deze laag is uitsluitend verantwoordelijk voor de initiële configuratie en het opstartproces van het systeem. [32] Deze laag is de centrale plaats waar afhankelijkheden worden samengesteld en waarin de toepassing in een geldige beginsituatie wordt gebracht.

Binnen Unity bestaat een alternatieve benadering via de service locator, een mechanisme dat een rol vervult die functioneel vergelijkbaar is met dependency injection. [39] Unity heeft geen centrale plaats om een cliënt op te starten waardoor er geen main laag mogelijk is. Ter vervanging bestaat een servicelocator die de rol hiervan kan overnemen. Dit heeft wel een nadeel, klassen die je wilt instantiëren via de service locator moeten overerven van Monobehaviour [40] wat zou zorgen voor een koppeling tussen elke laag met Unity. Dit werd in Ironclad Arena vermeden door een eigen service locator te schrijven die werkt op dezelfde manier als traditionele dependency injection.

5.5 Use cases

Binnen de software architectuur vervult de service-laag een centrale rol als schakel tussen het domein en de externe infrastructuurcomponenten. Een service laag zal namelijk alles samenbrengen om één actie uit te voeren binnen een softwaresysteem. De service-laag orkestreert daarbij de interacties tussen domeinmodellen en technische voorzieningen zoals databanken, messaging-systemen of externe API's, zonder dat domeinlogica afhankelijk wordt van infrastructuur details.

Volgens de principes van Clean Architecture [32], en specifiek het concept van screaming architecture, geef je elke actie die je kan ondernemen zijn eigen service. Dit zorgt ervoor dat elke use case zeer gefragmenteerd is. Voor een kaartspel waar elke use case meerdere gecorreleerde acties bevat is het nuttiger om meerdere acties te ondersteunen op één service.

5.6 Conclusie

Het kaartspel bevat vier lagen beginnende vanuit de domein laag waar we de centrale logica van het spel schrijven. Om dit domein te kunnen gebruiken leggen we hier een Service laag boven, deze laag legt vast wat allemaal nodig is om een actie binnen het systeem echt uit te voeren van start tot einde. Hiervoor leggen we vast welke infrastructuur allemaal nodig hebben voor elke service. Om de externe componenten hun implementaties vast te leggen gebruiken we een aparte laag genaamd ‘infrastructure’. Als laatste configureert en start het programma binnen de main laag.

6 Game engines

Het selecteren van een geschikte game engine wordt gebaseerd op twee criteria:

1. Netwerkfunctionaliteit: de mate waarin de engine ondersteuning biedt voor multiplayer en client-servercommunicatie.
2. Minimale overhead: de engine moet licht genoeg zijn om efficiënt te kunnen werken, zeker gezien de beperkte grafische vereisten van het project

Daarnaast is het relevant om onderscheid te maken tussen 2D- en 3D-enginecapaciteiten, aangezien een 2D-omgeving voor dit project de meest efficiënte keuze is binnen de beschikbare tijd.

6.1 Unity

Unity is een van de meest gebruikte game engines in de industrie en wordt vaak als eerste genoemd in vergelijkingen van gameframeworks. Unity beschikt over een uitgebreide basisarchitectuur die ondersteund wordt door een bijzonder grote catalogus aan plugins en packages. Deze modulariteit maakt het mogelijk om Unity in te zetten voor uiteenlopende toepassingen zonder dat de engine te complex of zwaar wordt. [41]

6.1.1 Functionaliteit

- Grafische mogelijkheden: Unity staat vooral bekend om zijn sterke 3D-capaciteiten, maar beschikt ook over robuuste 2D-functionaliteit, inclusief lighting systems, particles en tilemaps.
- Netwerkfunctionaliteit: Unity biedt een goed uitgebouwd multiplayer-ecosysteem. De Unity Multiplayer Center helpt bij het configureren van netcode door op basis van speltype en architectuur automatisch geschikte netwerkpakketten aan te bevelen.
- Documentatie en community: Dankzij de omvangrijke documentatie, community en fora is vrijwel elk probleem gedocumenteerd of elders besproken.
- Programmeertaal: Unity gebruikt C#, al verloopt projectstructuuropbouw anders dan bij een standaard .NET-oplossing.

6.1.2 Beoordeling

Unity is een uitstekende keuze voor ons project. Het biedt sterke netwerkondersteuning, flexibiliteit in grafische stijl (2D, 3D of een combinatie) en een rijke bron aan leermateriaal en ondersteuning. Deze kenmerken bieden veel vrijheid in architectuur en versnellen daarom de ontwikkeling.

6.2 Godot

Godot is een relatief jonge engine, maar heeft de voorbije jaren snel aan populariteit gewonnen. Ondanks zijn recente oorsprong beschikt de engine al over uitgebreide documentatie en een groeiend ecosysteem van tutorials en discussies. [42]

6.2.1 Functionaliteit

- Architectuur: Godot maakt gebruik van een node-based structuur waarin gameobjecten communiceren via signalen. Dit abstraheert een groot deel van de eventafhandeling en leidt tot een overzichtelijke en modulaire codebasis.
- Object Oriented Programming: Game-entiteiten volgens klassieke OOP-principes, zoals inheritance en composition
- 2D- en 3D-support: Godot blinkt uit in 2D, maar biedt ook degelijke 3D-capaciteiten. Dat is meer dan voldoende voor eenvoudige projecten.
- Programmeertalen: Ondersteuning voor C++, C# en GDScript, waarbij GDScript een Python-achtige taal is die nauw aansluit op Godots interne systemen.
- Deployability: Wide-platform deployment is mogelijk, zij het met meer configuratiewerk dan bij Unity.

6.2.2 Netwerkfunctionaliteit

Godot biedt een high-level netwerk-API, maar deze veronderstelt dat zowel client als server in Godot ontwikkeld worden. Omdat wij onze server engine-onafhankelijk willen houden, biedt Godot minder voordeel op netwerkvlak. Zelf netcode schrijven is mogelijk, maar vraagt aanzienlijk meer werk.

6.2.3 Beoordeling

Godot is aantrekkelijk door zijn compacte architectuur en lage overhead, maar de beperkingen op het gebied van serverintegratie maken het minder geschikt voor onze doelstellingen.

6.3 LibGDX

LibGDX is een open-source engine met een sterke focus op cross-platformontwikkeling. De engine kan spellen bouwen voor desktop, Android, iOS en web vanuit één codebase. [43]

6.3.1 Functionaliteit

- Platformondersteuning: Cross-platfrommogelijkheden zijn een sterke troef, maar de engine is specifiek geoptimaliseerd voor Android.
- Grafische mogelijkheden: Geschikt voor 2D en eenvoudige 3D, maar minder veelzijdig dan Unity of Godot.
- Programmeertaal: Java vormt de kern van LibGDX, waardoor de client ook geschreven moet worden in Java.

6.3.2 Netwerkfunctionaliteit

Op netwerkvlak biedt LibGDX weinig ingebouwde ondersteuning. Het grootste deel van de netcode moet zelf geschreven worden wat de ontwikkelingskosten verhoogt.

6.3.3 Beoordeling

Hoewel LibGDX krachtig is voor mobiele games, biedt het weinig voordelen voor dit project. De nadruk op Android en het gebrek aan netwerkfeatures maken het geen ideale keuze.

LibGDX legt ook een ongewenste keuze op de programmeertaal waardoor deze engine minder geschikt is voor Ironclad Arena.

6.4 Clickteam Fusion

Clickteam Fusion is een engine die vooral gericht is op eenvoud en visuele ontwikkeling. Het laat gebruikers spelmechanieken definiëren via events in plaats van via traditionele code. [44] [45]

6.4.1 Functionaliteit

- Toegankelijkheid: Ontwikkeling gebeurt grotendeels via event-gedreven regels, wat drempelloos is voor beginners.
- Programmatie: Mogelijkheid tot custom code, maar beperktere flexibiliteit dan in klassieke engines.

6.4.2 Netwerkfunktionaliteit

Standaard biedt Clickteam Fusion nauwelijks ondersteuning voor netwerken. Lacewing, een externe netwerkoplossing specifiek voor Fusion, kan gebruikt worden, maar introduceert een afhankelijkheid van een derde partij. Als de externe service uitvalt, wordt de game onbespeelbaar.

6.4.3 Beoordeling

Clickteam Fusion is niet geschikt voor een project waarbij de kernlogica zich op een externe server bevindt en waarbij netwerkbetrouwbaarheid cruciaal is. De engine fungeert in dit scenario enkel als grafische client, zonder noemenswaardige voordelen.

6.5 Photon Engine

Photon Engine biedt cloudgebaseerde multiplayeroplossingen en real-time netcode voor games. Het is krachtig, maar introduceert een afhankelijkheid van externe servers, licentiemodellen en vendor lock-in. Voor een academisch project waar volledige controle over architectuur belangrijk is, is Photon dus minder geschikt. [46]

6.6 Unreal Engine

Unreal Engine is een van de meest krachtige en uitgebreide engines in de industrie en wordt veel gebruikt bij AAA-studio's en hoogwaardige indieprojecten. [47] [48]

6.6.1 Functionaliteit

- Grafische capaciteiten: Unreal excelleert in geavanceerde rendering, realistische belichting en grote 3D-omgevingen.
- Netwerkfunktionaliteit: De engine beschikt over sterke ingebouwde netcode en ondersteunt het opzetten van dedicated servers.
- Documentatie: Uitgebreid en professioneel ondersteund.

6.6.2 Beoordeling

Ondanks de sterke technische mogelijkheden is Unreal Engine minder geschikt voor dit project. Het is ontworpen voor grafisch intensieve ervaringen, terwijl onze game vooral technisch van aard is en eenvoudige visuele vereisten heeft. De overhead van Unreal is dan ook te groot voor een lichte clientimplementatie.

6.7 Conclusie

Op basis van de analyse van verschillende game engines blijkt Unity de meest geschikte keuze voor dit project. Verschillende teamleden hebben al ervaring met deze engine, wat het ontwikkelproces versnelt. Unity biedt uitgebreide 2D- en 3D-functionaliteit, robuuste netwerkondersteuning en een rijk ecosysteem van documentatie en plugins. Dankzij deze veelzijdigheid hoeft geen netwerkcode compleet uniek opgebouwd te worden omdat de netcode gebaseerd is op bestaande, goed gedocumenteerde oplossingen. Dit maakt Unity de meest efficiënte, toegankelijke en toekomstbestendige keuze voor de ontwikkeling van onze gameclient.

7 Regels voor het kaartspel

7.1 Ontwikkeling van de spelregels

Er bestaan verschillende populaire kaartspellen, zoals Hearthstone, Magic: The Gathering, Yu-Gi-Oh! en Marvel Snap. Op basis van een vergelijkende analyse van deze spellen blijkt dat een combinatie van elementen uit Hearthstone en Yu-Gi-Oh! het meest geschikt is als basis voor het ontwerp van Ironclad Arena. Beide spellen kenmerken zich door toegankelijke maar tegelijk creatieve regelsets, wat ze bijzonder geschikt maakt om te integreren in een nieuw spelsysteem.

Dit onderzoek biedt een duidelijk beeld over de regels uit elk spel die werden overgenomen. De gevechtsmechanieken die betrekking hebben op rijen, kolommen, win- en verliesvoorwaarden en kaarttypes zijn geïnspireerd door Yu-Gi-Oh!. De basisconcepten, zoals de opdeling in klassen, decks van 30 kaarten en het gebruik van sleutelwoorden op kaarten, zijn gemodelleerd naar Hearthstone.

Om de regels consistent en coherent te houden, werd een selectie gemaakt van relevante regelcomponenten uit beide spellen. Vervolgens werd AI ingezet om te evalueren of deze regels logisch op elkaar aansluiten en waar nodig aanvullende voorstellen te genereren.

7.2 Ontwikkeling van de kaarten

Het spel hanteert een Steampunk-thema, wat als uitgangspunt diende voor het ontwerp van de kaarten. Om een werkend en gevarieerd spel te ontwikkelen, werd beslist om 100 unieke kaarten te creëren: 50 neutrale kaarten en 50 klasse specifieke kaarten.

De neutrale kaarten bestaan uit 38 minionkaarten en 12 spreukkaarten. De klasse specifieke kaarten zijn verdeeld over vijf heldenklassen; voor elke held werden tien unieke kaarten ontworpen, bestaande uit zes minionkaarten en vier spreukkaarten. Deze kaarten zijn aanwezig in bijlage 1.

Voor de initiële generatie van deze kaarten werd AI gebruikt. In de prompt werd uitgebreide context meegegeven, waaronder de spelmechanieken, het thematische kader, het gewenste aantal kaarten en hun verdeling, de minimum- en maximumwaarden voor mana, aanval en 'life points', en het zeldzaamheidssysteem. De gegenereerde resultaten werden nadien verfijnd en aangepast om volledig te voldoen aan de ontwerpdoelen, maar vormden een solide basis voor verdere ontwikkeling. Ten slotte kreeg elke kaart een schets voor een unieke weergave binnen het spel.

7.3 Regels

7.3.1 Spelopzet

Het spel wordt gespeeld door twee spelers. Elke speler beschikt over een deck van 30 kaarten en een held. Deze decks mogen uitsluitend bestaan uit neutrale kaarten en kaarten die behoren tot de klasse van de gekozen held.

7.3.2 Kaarttypes en eigenschappen

Elke kaart heeft een manakost, een hoeveelheid life points, een attack-waarde en eventueel een extra vaardigheid. Wanneer de life points van een kaart tot nul worden herleid, wordt de kaart vernietigd en van het speelveld verwijderd. Vernietigde kaarten mogen niet terug in het deck geplaatst worden en worden op de aflegstapel gelegd.

Afhankelijk van het type kaart kan een kaart andere kaarten aanvallen of een effect uitoefenen op het speelveld. Daarnaast bestaan er heldenkaarten. Deze kaarten kunnen niet aanvallen, maar beschikken wel over life points en kunnen speciale effecten activeren binnen het spel. Ten slotte zijn er spreukkaarten. Spreukkaarten worden niet op het veld geplaatst, maar éénmalig gespeeld door ze te selecteren. Na activatie voeren ze onmiddellijk een specifiek effect uit.

7.3.3 Klassen

Binnen het spel bestaan er vijf klassen. Elke klasse beschikt over unieke kaarten die niet door andere klassen kunnen worden gebruikt. Iedere klasse wordt voorgesteld door één of meerdere helden. De beschikbare klassen zijn: warrior, mage, priester, hunter en rogue.

- De warrior is een eenvoudige held die vooral gebruikmaakt van sterke kaarten met weinig speciale effecten.
- De mage gebruikt elementen om te vechten en beschikt over veel kaarten met speciale effecten die invloed hebben op het veld.
- De priester focust op het in leven houden van zijn kaarten en versterkt de kaarten op zijn veld.
- De hunter maakt gebruik van wilde beesten en valstrikken. Hij kan valstrikken plaatsen op zijn kaarten, waardoor zijn tegenstander gestraft wordt bij het aanvallen.
- De rogue maakt gebruik van misleiding en kan kaarten uitwisselen met zijn tegenstander.

7.3.4 Start van het spel en wincondities

Bij het begin van het spel ontvangen beide spelers drie willekeurige kaarten uit hun deck. Deze kaarten kunnen worden uitgewisseld voor nieuwe, waarbij de speler zelf kiest welke kaarten hij wil vervangen.

Beide spelers starten met 8000 life points en 0 mana. Een speler verliest het spel wanneer zijn life points tot nul worden gereduceerd of wanneer hij geen kaarten meer heeft op zowel het veld als in zijn hand.

Als beide spelers geen kaarten meer hebben, wordt de winnaar bepaald op basis van de resterende life points. Wanneer beide spelers in dat geval een gelijk aantal life points hebben, eindigt het spel in een gelijkspel (draw).

7.3.5 Het speelveld

Het speelveld bestaat uit vier rijen, waarvan twee rijen per speler. Elke rij bevat vijf vakjes waarin kaarten geplaatst kunnen worden. Elke speler heeft twee rijen, een voorste en achterste rij.

- Op de voorste rij kunnen uitsluitend strijderskaarten worden geplaatst. Deze kaarten kunnen andere kaarten of de held aanvallen.
- Op de achterste rij kunnen kaarten worden geplaatst die ondersteuning bieden, op afstand aanvallen of een effect uitoefenen op het veld.

De voorste rij dient als een muur tegen aanvallen naar de held of de achterste rij. Enkel wanneer deze rij leeg is kunnen de achterliggende kaarten en held worden aangevallen.

7.3.6 Beurtverloop

Tijdens elke beurt moet een speler één nieuwe kaart trekken uit het deck en wordt de manalimiet met één verhoogd. De manalimiet kan maximaal tien bedragen. Aan het begin van elke beurt wordt de beschikbare mana volledig aangevuld.

Mana kan worden gebruikt om kaarten te spelen en om de unieke vaardigheid van de held te activeren. De speler mag zoveel kaarten spelen als zijn beschikbare mana toelaat.

7.3.7 Aanvallen

De speler mag iedere beurt één kaart gebruiken om aan te vallen. Deze kaart moet op het veld aanwezig zijn en moet kunnen aanvallen. De getroffen kaart zal zijn life points moeten aftrekken met de *attack* van de aanvallende kaart.

7.4 Regels uit andere games

Voor het ontwerp van de spelregels werd inspiratie gehaald uit bestaande succesvolle digitale kaartspellen. In het bijzonder werden HearthStone en Yu-Gi-Oh! geanalyseerd. Beide spellen hanteren verschillende ontwerp filosofieën en mechanieken die waardevolle inzichten bieden voor de ontwikkeling van een eigen spel.

7.4.1 HearthStone

HearthStone is een digitaal kaartspel dat wordt gespeeld door twee spelers, waarbij elke speler gebruikmaakt van een deck van 30 kaarten. Het doel van het spel is om de tegenstander te verslaan door de life points van zijn held tot nul te reduceren. Elke held start met een vaste hoeveelheid life points.

Het spel verloopt beurtelings. Aan het begin van elke beurt trekt de actieve speler één kaart, met een maximale handgrootte van 10 kaarten. Spelers maken gebruik van een manasysteem om kaarten te spelen. Aan het begin van elke beurt ontvangt de speler één extra mana kristal, die automatisch wordt aangevuld. Het maximumaantal mana bedraagt tien.

Mana wordt gebruikt om verschillende soorten kaarten te spelen, waaronder minions en spreuken, of om wapens uit te rusten. Daarnaast kan de speler tijdens zijn beurt minions laten aanvallen of de unieke Hero Power van zijn held activeren, zolang hij over voldoende mana beschikt.

Gevechten worden altijd geïnitieerd door de aanvallende speler, die zowel het aanvallende personage als het doelwit selecteert. Minions brengen schade toe gelijk aan hun attack-waarde. Helden kunnen eveneens aanvallen als zij een wapen hebben uitgerust. Zowel minions als helden kunnen doelwit zijn van aanvallen. De toegebrachte schade vermindert rechtsreeks de life points van het doelwit, waarbij speciale vaardigheden of effecten de schade kunnen beïnvloeden. [49]

7.4.2 Yu-Gi-Oh!

Yu-Gi-Oh! is oorspronkelijk een fysiek ruilkaartspel en verschilt aanzienlijk van HearthStone in complexiteit en structuur. Elke speler beschikt over een hoofddeck van 40 tot 60 kaarten en kan daarnaast een *extra deck* gebruiken dat tot 15 speciale kaarten kan bevatten. Optioneel is er ook een deck dat je toestaat om kaarten te wisselen tussen duels, dit noemen we ook wel een *side deck* en mag 15 kaarten bevatten.

Aan het begin van een duel worden de decks geschud en trekken beide spelers vijf kaarten. Vervolgens wordt bepaald welke speler begint. Het spel is opgebouwd uit duidelijk gedefinieerde fasen per beurt: de Draw Phase, Standby Phase, Main Phase 1, Battle Phase, Main Phase 2 en End Phase. De speler die als eerste begint, slaat zijn eerste Draw Phase over.

Het oproepen van monsters vormt een centraal onderdeel van het spel. Een speler mag per beurt één monster normaal oproepen, hetzij in aanvalshouding, hetzij in verdedigingshouding. Sterkere monsters vereisen offers (tributes): monsters van niveau vijf of zes vereisen één offer, terwijl monsters van niveau zeven of hoger twee offers vereisen. Daarnaast bestaan er speciale monsters die via Special Summons op het veld kunnen worden geplaatst onder specifieke voorwaarden.

Yu-Gi-Oh! kent drie hoofdtypes kaarten: monsterkaarten, spreukkaarten en valkaarten. Monsterkaarten worden gebruikt om aan te vallen of te verdedigen. Spreukkaarten kunnen tijdens de beurt van de speler geactiveerd worden en hebben uiteenlopende effecten. Valkaarten moeten eerst gedekt op het veld worden geplaatst en kunnen pas in een latere beurt, vaak tijdens de beurt van de tegenstander, geactiveerd worden.

Gevechten vinden plaats tijdens de Battle Phase. Wanneer een monster in aanvalshouding een ander monster in aanvalshouding aanvalt wordt het monster met de laagste aanval vernietigd en ontvangt de eigenaar het verschil in aanvalskracht als schade aan zijn levenspunten. Bij een aanval op een monster in verdedigingshouding wordt enkel het verdedigende monster vernietigd als de aanvalskracht hoger is dan de verdedigingswaarde; levenspunten worden in dat geval niet verminderd. Als de verdediging hoger is, ontvangt de aanvaller schade. Wanneer de tegenstander geen monsters op het veld heeft, kan een monster rechtsreeks de life points van de tegenstander aanvallen. Het spel wordt gewonnen door de speler die als eerste de life points van zijn tegenstander tot nul herleidt. [50] [51] [52] [53]

8 Game design workflow

Binnen game development wordt doorgaans gewerkt met verschillende ontwerpdocumenten die het ontwikkelproces structureren en ondersteunen. Twee veelvoorkomende voorbeelden hiervan zijn een art bible en een game designdocument. Deze documenten fungeren als leidraad voor het ontwikkelteam en zorgen voor consistentie tijdens de productie van het spel.

8.1 Art bible

Een art bible dient als een enkele punt van waarheid rond de visuele elementen binnen het spel. Dit document wordt normaal gezien gemaakt door een paar artiesten, typisch deze die de leiding hebben. De andere artiesten gebruiken dan dit document doorheen het project om ervoor te zorgen dat het spel visueel uniform is.

Verschillende onderdelen worden aangekaart binnen het document: bedoelde atmosfeer, vormen, kleuren, gebruikersinterface, technische details. Voor alle onderdelen worden er voorbeelden gegeven van wat juist is en wat verkeerd is. [54]

Voor dit project is een art bible echter niet nodig. De focus ligt meer op het technische en minder op het visuele. Aangezien er ook geen team van artiesten is zou de art bible niet benut kunnen worden. De art bible dient vooral om andere teamleden te begeleiden, als er maar één artiest is, dan is het redelijk overbodig.

8.2 Game designdocument

Typisch start een game designdocument (GDD) met een beschrijving van de kern van het spel. Dit is een beknopte samenvatting die uitlegt waar het spel om draait en welke handelingen de speler uitvoert. Vervolgens worden de design pilaren gedefinieerd. Dit zijn een beperkte aantal kernconcepten of thematische principes die richting geven aan alle ontwerpbeslissingen binnen het spel.

Daarna worden de verschillende onderdelen van het spel gedetailleerd uitgewerkt. Dit omvat onder meer de spelmechanieken, levelstructuren en andere relevante systemen. Het doel is om het volledige spel, van begin tot einde, zo volledig mogelijk te beschrijven. Een lezer moet op basis van dit document een duidelijk en volledig beeld krijgen van het eindproduct. Door deze aanpak kunnen ontwikkelaars zelfstandig en parallel aan verschillende onderdelen werken, wat de kans op miscommunicatie en latere ontwerpdiscussies verkleint.

Daarnaast bevat een GDD doorgaans ook een sectie over het doelpubliek. In dit onderdeel wordt geschreven voor welk type speler het spel bedoeld is, rekening houdend met factoren zoals ervaring, interesses en speelstijl. Dit helpt om ontwerpkeuzes beter af te stemmen op de beoogde gebruikersgroep.

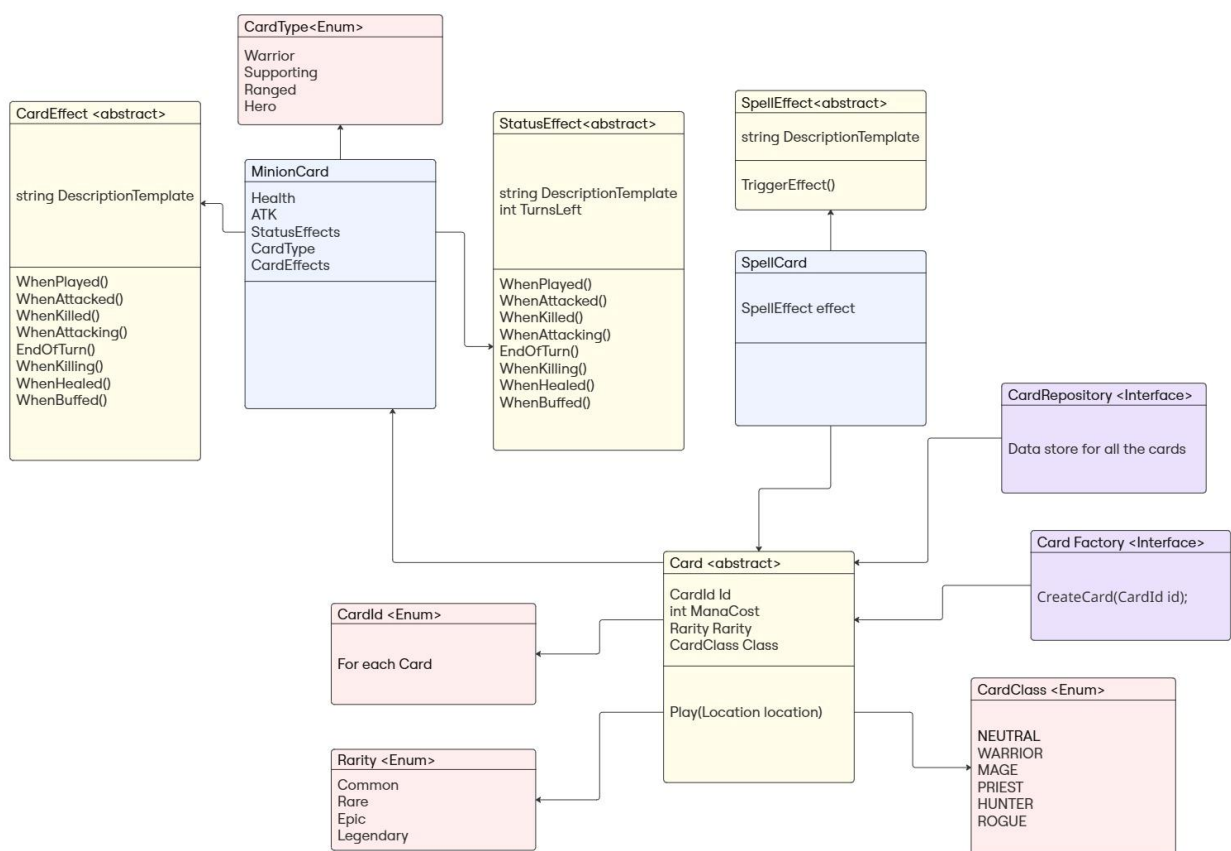
Voor dit project werd echter besloten om geen volledig game designdocument uit te werken. Gezien de beperkte beschikbare tijd en de focus van het project zou het opstellen van een uitgebreid GDD onvoldoende meerwaarde bieden. Hoewel een dergelijk document interessant en leerzaam is, werd de keuze gemaakt om de inspanningen te richten op de technische en architecturale aspecten van het spel. [55] [56]

9 Implementatie van kaarten en effecten

De implementatie van kaarten en hun bijbehorende effecten vormt een belangrijk onderdeel van het ontwerp en de ontwikkeling van de online trading card game Ironclad Arena. Kaarten bepalen niet alleen welke acties binnen het spel mogelijk zijn, maar dragen ook bij aan de strategische diepgang en variatie. Een robuuste en uitbreidbare architectuur voor kaarten en effecten is daarom cruciaal om zowel huidige als toekomstige kaarten te kunnen ondersteunen.

In bestaande implementaties van kaartspellen wordt vaak gewerkt met grote klassen die te veel code bevatten. [57] [58] Deze aanpak leidt doorgaans tot moeilijk onderhoudbare code, beperkte uitbreidbaarheid en een verhoogd risico op regressiefouten. Om deze problemen te vermijden, werd binnen dit project bewust gekozen voor een modulaire en domeingedreven benadering. Deze keuze heeft als doel architecturale valkuilen te vermijden en maximale flexibiliteit te behouden.

9.1 Structuur en inhoud van kaarten



Figuur 9.1 Structuur van kaarten

Kaarten vormen de centrale verzamelobjecten binnen Ironclad Arena. Elke kaart bevat alle informatie die nodig is om haar rol binnen het spel correct te vervullen. Dit omvat onder meer basiskennmerken zoals aanvalskracht, levenspunten, een naam, een visuele representatie, een kaarttype en de logica die het specifieke gedrag van de kaart definieert. Figuur 9.1

bevat een grafische voorstelling die de structuur van de kaarten kan toelichten, in dit hoofdstuk zal elk van deze elementen aan bod komen.

9.1.1 Opslag van kaartgegevens

Statische kaartgegevens, zoals namen, statistieken en afbeeldingen, lenen zich goed voor persistente opslag in een databank. Deze gegevens wijzigen zelden en kunnen efficiënt beheerd worden via traditionele opslagmechanismen.

De opslag van kaarteffecten vormt daarentegen een grotere uitdaging. Effecten bevatten gedragslogica die contextafhankelijk is en sterk kan variëren tussen kaarten.

In plaats van een rigide databankschema te ontwerpen om deze logica op te slaan, werd gekozen voor een repository-gedreven aanpak binnen een ports-and-adapters-architectuur. Hierdoor blijft de domeinlaag losgekoppeld van de concrete opslagtechnologie, wat de testbaarheid en vervangbaarheid van infrastructuurcomponenten ten goede komt.

9.1.1.1 Domain specific language

Een mogelijke aanpak voor het modelleren van kaarteffecten is het gebruik van een Domain Specific Language (DSL). In deze benadering wordt de logica van een effect opgeslagen als tekst en tijdens runtime geïnterpreteerd, bijvoorbeeld via het Interpreter pattern. Hoewel deze aanpak een hoge mate van flexibiliteit biedt, brengt zij ook aanzienlijke nadelen met zich mee. Het ontwerpen, implementeren en onderhouden van een DSL vereist een hoge ontwikkelkost en verhoogt de complexiteit van het systeem aanzienlijk. Daarnaast introduceert runtime-interpretatie extra overhead en foutgevoeligheid.

9.1.1.2 Gebruik van de Roslyn Scripting API

Om bovenstaande nadelen te vermijden, maakt Ironclad Arena gebruik van de Roslyn Scripting API. Deze externe bibliotheek maakt het mogelijk om C#-code dynamisch uit te voeren vanuit een bestaande C#-context. De logica van kaarteffecten wordt hierbij opgeslagen als C#-code in tekstvorm en uitgevoerd via de EvaluateAsync-methode. [59]

Deze aanpak biedt meerdere voordelen. Ontwikkelaars kunnen zo alle effecten schrijven in een algemeen bekende programmeertaal, waardoor de leercurve van een DSL vermeden wordt. Bovendien worden de ontwikkelingskosten en onderhoudslast van een eigen scriptingtaal geëlimineerd door gebruik te maken van een bestaande, mature oplossing. [60] [61]

9.1.1.3 Gebruik van kaart-ID's

Elke kaart binnen het spel beschikt over een unieke identifier. Deze identifier fungeert als primair referentiepunt binnen zowel de client- als serverarchitectuur. Door uitsluitend de kaart-ID naar de client te communiceren, kan overbodige overdracht van statische kaartgegevens worden vermeden. Dit resulteert in efficiëntere netwerkcommunicatie en een duidelijke scheiding tussen representatie en logica.

9.1.1.4 Instantiatie

Aangezien kaarteffecten worden opgeslagen als uitvoerbare code in tekstvorm, is het instantiatieproces van kaarten complexer dan bij een puur datagedreven model. Om dit proces te centraliseren en te structureren, wordt gebruikgemaakt van een CardFactory. Deze klasse is verantwoordelijk voor het aanmaken van kaartinstanties en het correct koppelen van effecten aan hun bijbehorende kaartobjecten. Hierdoor wordt gegarandeerd dat elke kaart op een consistente manier wordt geïnitialiseerd voordat zij deelneemt aan het spelverloop.

9.1.2 Heroes

Heroes worden binnen Ironclad Arena gemodelleerd als een specifieke vorm van kaart. In tegenstelling tot reguliere kaarten beschikken heroes niet over manakost en aanvalskracht. Hun primaire functie is het bijhouden van de levenspunten van de speler. Door heroes te implementeren als kaarten kunnen zij gebruikmaken van dezelfde logica, eigenschappen en infrastructuur als andere kaarttypes.

Daarnaast laat deze aanpak toe om hero-abilities op een natuurlijke manier te integreren binnen het bestaande effectensysteem. Hoewel hero-abilities in de huidige implementatie conceptueel zijn gehouden, biedt de gekozen architectuur voldoende ruimte om deze in de toekomst uit te breiden zonder fundamentele wijzigingen aan het domeinmodel.

9.2 De Game-klasse als kerncomponent

Naast kaarten vereist het spel een centrale context waarin alle interacties plaatsvinden. Deze rol wordt vervuld door de Game-klasse, die fungeert als het primaire aanspreekpunt voor alle acties en gebeurtenissen tijdens een spel. Het beheert de speltoestand, coördineert interacties tussen objecten en bewaakt het naleven van spelregels.

9.2.1 Gebruik van events

Om spelgebeurtenissen te communiceren naar de client, wordt een event-gedreven aanpak gehanteerd. Elke relevante actie binnen het spel resulteert in één of meerdere events die verzameld en doorgestuurd worden via de infrastructuurlaag.

9.2.1.1 Eventbus

Bij het aanmaken van een spelinstantie wordt een eventbus geïnjecteerd. De interface van deze eventbus wordt gedefinieerd in de domeinlaag, terwijl de concrete implementatie zich in de infrastructuurlaag bevindt. Vanuit het perspectief van de domeinlaag is enkel het concept van een eventbus relevant; de uiteindelijke bestemming van de events is ondergeschikt. Deze abstractielaag draagt bij aan een lage koppeling en verhoogde flexibiliteit.

9.2.1.2 Generieke events

Events worden zo generiek mogelijk gehouden om hergebruik en uitbreidbaarheid te bevorderen. Een event zoals CardAttacked kan bijvoorbeeld worden gebruikt voor aanvallen door minions, spells of andere effecten. Door de oorzaak van een gebeurtenis los te

koppelen van het event zelf, wordt voorkomen dat toekomstige uitbreidingen beperkt worden door eerdere ontwerpkeuzes.

9.2.2 Effectensysteem

Effecten vormen een van de belangrijkste bouwstenen van een trading card game. Zij bepalen het unieke gedrag van kaarten en maken complexe interacties mogelijk. Het domeinmodel van Ironclad Arena is daarom expliciet ontworpen om een grote variëteit aan effecten te ondersteunen.

Het Strategy design pattern wordt toegepast om dynamisch gedrag aan kaarten toe te voegen. Effecten worden hierbij gemodelleerd als afzonderlijke objecten die via interfaces aan kaarten gekoppeld worden.

9.2.2.1 Soorten Effecten

Effecten kunnen op verschillende plaatsen worden aangesproken. Hier bespreken we elk van de soorten effecten die kunnen voorkomen.

9.2.2.1.1 Spreuk effecten

Spreuk effecten worden geactiveerd wanneer een spreukkaart wordt gespeeld. Deze effecten worden exact één keer uitgevoerd en implementeren een eenvoudige interface met een enkele uitvoeringsmethode.

9.2.2.1.2 Kaart effecten

Elke kaart kan inherente effecten bevatten die verbonden zijn aan specifieke triggers, zoals het betreden van het speelveld of het uitvoeren van een aanval. Deze effecten vertegenwoordigen permanente of conditionele eigenschappen die uniek zijn voor de kaart.

9.2.2.1.3 Status Effecten

Status effecten worden gebruikt voor tijdelijke of doorlopende effecten die door kaarten aan elkaar worden opgelegd. Een minion kan meerdere status effecten tegelijk bezitten. Deze effecten maken gebruik van dezelfde triggers als card effecten. Dit maakt het mogelijk om tijdelijke effecten te maken, of effecten af te spelen op andere kaarten.

Door status effecten ook aan heroes toe te kennen, kunnen globale effecten worden geïmplementeerd die invloed hebben op alle kaarten van een speler.

9.2.2.2 Triggers of hooks

Het spel definieert acht vaste momenten waarop effecten geactiveerd kunnen worden. Card effecten en status effecten kunnen één of meerdere van deze hooks implementeren. Op deze manier kan effectlogica nauwkeurig inspelen op specifieke fases of gebeurtenissen binnen het spelverloop.

9.2.2.3 Contextuele informatie bij triggers

Bij het activeren van een effect is vaak aanvullende context nodig over de gebeurtenis die het effect heeft veroorzaakt. Zo kan bij een aanval relevant zijn welke kaart de aanvaller of verdediger is. Deze informatie wordt meegegeven aan de trigger, zodat effecten voldoende context hebben om correct te functioneren.

9.2.3 Architecturale keuzes rond de Game-klasse

De Game-klasse is een omvangrijke klasse waarin een groot deel van de centrale spellogica samenkomt. Het is daarom essentieel om binnen deze klasse bewuste architecturale keuzes te maken om ontwerpfouten te vermijden.

9.2.3.1 Centrale afhandeling van events

Door alle significante spelacties via de Game-klasse te laten verlopen, hoeft enkel deze klasse kennis te hebben van het eventsysteem. Alternatieve oplossingen, waarbij events vanuit verschillende componenten worden afgevuurd, leiden tot verhoogde complexiteit en verminderde controle over de speltoestand.

9.2.3.2 Open informatie, gesloten gedrag

Effecten krijgen toegang tot alle benodigde informatie, maar mogen het spel uitsluitend beïnvloeden via expliciete methoden van de Game-klasse. Dit creëert een centraal controlepunt om na te gaan of spelregels correct worden nageleefd, terwijl effecten toch voldoende flexibiliteit behouden.

9.2.3.3 Partial classes

Om de omvang en complexiteit van de Game-klasse beheersbaar te houden, wordt deze opgesplitst in meerdere partial classes verspreid over verschillende bronbestanden. Hoewel deze aanpak formeel afwijkt van het Single Responsibility Principle, blijkt het in de praktijk noodzakelijk gezien de centrale rol van de Game-klasse binnen het systeem.

9.3 Users & Decks

Het beheer van gebruikers en hun kaartencollecties vormt een relatief eenvoudig onderdeel van het systeem. De interne werking vertoont sterke gelijkenissen met een klassieke CRUD-architectuur. Hoewel er specifieke spelregels van toepassing zijn zoals het nakijken of een deck geldig is, vereisen deze geen complexe softwarearchitectuur patronen en kunnen zij op een conventionele manier worden afgedwongen.

9.4 Conclusie

Door te kiezen voor een flexibel en modulair domeinmodel biedt Ironclad Arena een solide basis voor de implementatie van complexe kaarteffecten en spelmechanieken. De combinatie van een centrale gamearchitectuur, een generiek eventsysteem en een uitbreidbaar effectensysteem maakt het mogelijk om met minimale wijzigingen veel variatie aan spelgedrag te ondersteunen. Deze aanpak draagt bij aan de onderhoudbaarheid, uitbreidbaarheid en robuustheid van het systeem op lange termijn.

10 Conclusie

Deze bachelorproef toont aan dat het mogelijk is een online trading card game te ontwikkelen die zowel modulaair als onderhoudbaar is, mits weloverwogen architecturale en technologische keuzes. Binnen dit project werd Ironclad Arena gerealiseerd als een proof of concept waarin deze principes werden toegepast.

Een eerste essentiële ontwerpoverweging betreft de netwerkarchitectuur. Voor Ironclad Arena werd gekozen voor een client-servermodel, aangezien dit model voordelen biedt op het vlak van beveiliging, matchmaking en synchronisatie van de speltoestand. Door de centrale rol van de server kan de gamestate op een gecontroleerde manier worden beheerd, wat de kans op inconsistenties aanzienlijk vermindert.

Naast de architectuur is ook de communicatiemethode tussen client en server van belang voor performantie. Hoewel verschillende oplossingen beschikbaar zijn, maakt Ironclad Arena gebruik van een custom protocol gebaseerd op ENet. Deze aanpak maakt performante en efficiënte communicatie mogelijk door middel van op maat gemaakte pakketstructuren die zijn afgestemd op de specifieke noden van het spel.

Een doordachte softwarearchitectuur blijkt eveneens fundamenteel bij de ontwikkeling van een digitaal kaartspel. In dit project werd de code georganiseerd volgens principes uit Domain-Driven Design en het Ports & Adapters-patroon, wat resulteerde in een architectuur met vier horizontale lagen. Daarnaast werden verticale lagen geïntroduceerd op basis van use cases, wat de scheiding van verantwoordelijkheden en de uitbreidbaarheid van het systeem bevordert.

Game engines spelen een ondersteunende rol in het ontwikkelproces door generieke technische complexiteit te abstraheren. Ironclad Arena werd ontwikkeld met Unity, een veelgebruikte game engine die toegankelijke functionaliteiten en uitgebreide documentatie aanbiedt, waardoor de focus kon liggen op de spelmechanieken en architecturale keuzes.

Wat betreft de spelregels werd gekozen voor een eigen regelsysteem dat specifiek is ontworpen voor Ironclad Arena. Hierbij dienden bestaande trading card games zoals Yu-Gi-Oh! en Hearthstone als inspiratiebron, zonder hun regels rechtstreeks te kopiëren. Dit maakte het mogelijk om de spelmechanieken af te stemmen op de technische en ontwerpdoelstellingen van het project.

De implementatie van kaartspelregels brengt echter ook verschillende valkuilen met zich mee, zoals aangetoond in andere openbare projecten. Om onderhoudbaarheid en uitbreidbaarheid te garanderen, werd in Ironclad Arena kaart-specifieke logica ondergebracht in een Domain Specific Language. Daarnaast vervult de gameklasse een centrale rol binnen het systeem. Hoewel deze klasse meerdere verantwoordelijkheden draagt, biedt deze aanpak voordelen, waaronder vereenvoudigde eventafhandeling en een duidelijk centraal aanspreekpunt voor spelacties.

Samenvattend werd Ironclad Arena ontwikkeld met het oog op modulariteit, uitbreidbaarheid en onderhoudbaarheid. Alternatieveteem niet als perfect kan worden beschouwd en alternatieve ontwerpkeuzes mogelijk zijn, blijken de gemaakte beslissingen binnen de context van dit project geschikt en effectief voor de ontwikkeling van een online trading card game.

11 Referenties

- [1] Board Game Geek, „Collectible: Collectible Card Games | Family | BoardGameGeek,” Board Game Geek, 03 may 2024. [Online]. Available: <https://boardgamegeek.com/boardgamefamily/6040/collectible-collectible-card-games>. [Geopend 05 january 2026].
- [2] J. P. Williams, „Gaming as Culture: Essays on Reality, Identity and Experience in Fantasy Games,” 02 may 2007. [Online]. Available: <https://web.archive.org/web/20131202230411/http://www3.ntu.edu.sg/home/patrick.williams/PDFs/Williams%20-%20CSGs.pdf>. [Geopend 05 january 2026].
- [3] E. Ham, „Rarity and Power: Balance in Collectible Object Games,” *Game Studies*, vol. 10, nr. 1, 2010.
- [4] J. V. D. M. W. Brian David-Marshall, „Trading Card Game Industry,” SuperData Research Inc., New York, 2010.
- [5] T. Chalk, „20 Years Of Magic: The Gathering, A Game That Changed The World,” *sabotagetimes*, 31 july 2013. [Online]. Available: <https://web.archive.org/web/20140522124157/http://sabotagetimes.com/life/20-years-of-magic-the-gathering-a-game-that-changed-the-world/>. [Geopend 05 january 2025].
- [6] V. Ekanayake, „Getting started with the Pokemon Trading Card Game (TCG) for parents,” *virantha*, 06 july 2016. [Online]. Available: <https://virantha.com/2016/07/06/parent-guide-to-starting-pokemon/>. [Geopend 05 january 2026].
- [7] Konami Digital Entertainment, „Yu-Gi-Oh! Card Game Breaks Guinness Record with 25.1 Billion Cards Sold New Series Available Worldwide,” Konami Digital Entertainment, 14 june 2011. [Online]. Available: <https://web.archive.org/web/20110620054031/http://www.konami-digital-entertainment.co.jp/en/news/topics/2011/0614/>. [Geopend 05 january 2026].
- [8] R. J. Maly, „Comparison of Centralized,” ETH Zurich, Zurich, 2003.
- [9] G. Gambetta, „Fast-Paced Multiplayer (Part I): Client-Server Game Architecture,” *gabrielgambetta.com*, 23 August 2025. [Online]. Available: <https://www.gabrielgambetta.com/client-server-game-architecture.html#dealing-with-networks>. [Geopend 05 Januari 2026].
- [10] G. Kroupp, „Mastering Multiplayer Game Architecture: Choosing the Right Approach,” *getgud.io*, 22 July 2024. [Online]. Available: <https://www.getgud.io/blog/mastering-multiplayer-game-architecture-choosing-the-right-approach/>. [Geopend 05 Januari 2026].
- [11] M. Weilbacher, „Dedicated Servers in Gears of War 3,” in *GDC 2012*, San Francisco, California, 2012.
- [12] T. Ford, „'Overwatch' Gameplay Architecture and Netcode,” in *GDC 2017*, San Francisco, California, 2017.
- [13] S. Zander, „Dissecting server-discovery traffic patterns generated by multiplayer first person shooter games,” ACM Digital Library, New York, 2005.

- [14] C. J. S. Z. Grenville Armitage, „Topological Optimisation for Online First Person,” Swinburne University of Technology, Melbourne, Australia, 2012.
- [15] L. Apperson, „What is an API? An Essential Guide in Software and Game Development,” medium.com, 16 december 2023. [Online]. Available: <https://medium.com/@lemapp09/what-is-an-api-an-essential-guide-in-software-and-game-development-dc1a53ba72de>. [Geopend 05 januari 2026].
- [16] Akamai, „What Is an Event Bus?,” Akamai, 12 februari 2025. [Online]. Available: <https://www.akamai.com/glossary/what-is-an-event-bus>. [Geopend 05 januari 2026].
- [17] vertx.io, „Vert.x-Web,” vertx.io, 2026. [Online]. Available: <https://vertx.io/docs/vertx-web/java/>. [Geopend 05 januari 2026].
- [18] P. com.google.common.eventbus, „Package com.google.common.eventbus,” Google, 2010. [Online]. Available: <https://guava.dev/releases/19.0/api/docs/com/google/common/eventbus/package-summary.html>. [Geopend 05 januari 2026].
- [19] v. I. 3. einaros, „ws: a Node.js WebSocket library,” npm, 26 december 2023. [Online]. Available: <https://www.npmjs.com/package/ws>. [Geopend 05 januari 2026].
- [20] A. Myhre, „Unity-Event-Bus,” Adam Myhre, 29 october 2023. [Online]. Available: <https://github.com/adammyhre/Unity-Event-Bus>. [Geopend 05 januari 2026].
- [21] elpatrongames69, „Why don't you use eventbus?,” Unity, februari 2024. [Online]. Available: <https://discussions.unity.com/t/why-dont-you-use-eventbus/940577/3>. [Geopend 05 januari 2026].
- [22] __nostromo__, „Event systems over a network,” gamedev.net, 04 october 2022. [Online]. Available: <https://www.gamedev.net/forums/topic/713138-event-systems-over-a-network/>. [Geopend 05 januari 2026].
- [23] IBM, „What is REST API?,” IBM, 24 april 2025. [Online]. Available: <https://www.ibm.com/think/topics/rest-apis>. [Geopend 05 januari 2026].
- [24] user406126, „Authoritative vs REST (server),” Game development stack exchange, august 2021. [Online]. Available: <https://gamedev.stackexchange.com/questions/194672/authoritative-vs-rest-server>. [Geopend 05 januari 2026].
- [25] S. Neupane, „What is Polling ?,” Medium, 19 september 2024. [Online]. Available: <https://medium.com/@sankalpa115/what-is-polling-b1ff70e87001>. [Geopend 05 januari 2026].
- [26] C. "Battle(non)sense", „How netcode works, and what makes 'good' netcode,” PCGamer, 25 october 2017. [Online]. Available: <https://www.pcgamer.com/netcode-explained/>. [Geopend 05 januari 2026].
- [27] Unity, „Unity multiplayer overview,” Unity, 24 december 2025. [Online]. Available: <https://docs.unity3d.com/Manual/multiplayer-overview.html>. [Geopend 05 januari 2026].
- [28] Unity, „Build multiplayer games with Unity Netcode,” Unity, 2025. [Online]. Available: <https://unity.com/features/netcode>. [Geopend 05 januari 2026].

- [29] U. Engine, „Networking and Multiplayer,” Epic games, 28 may 2025. [Online]. Available: https://dev.epicgames.com/documentation/en-us/unreal-engine/networking-and-multiplayer-in-unreal-engine?application_version=5.6. [Geopend 05 january 2026].
- [30] pierre, „Beginner's Guide to Game Networking,” pvigier's blog, 08 september 2019. [Online]. Available: <https://pvigier.github.io/2019/09/08/beginner-guide-game-networking.html>. [Geopend 05 january 2026].
- [31] J. F. Bellinger, „NuGet Gallery | ENet 1.3.6.3,” Microsoft, 2011. [Online]. Available: <https://www.nuget.org/packages/ENet>. [Geopend 05 january 2026].
- [32] R. C. Martin, „Clean Architecture,” in *Clean Architecture A Craftsman's Guide to Software Structure and Design*, Hoboken, New Jersey, Prentice Hall/Pearson, 2017, pp. 111-132.
- [33] R. Nystrom, „Architecture, Performance, and Games,” gameprogrammingpatterns.com, 2021. [Online]. Available: <https://gameprogrammingpatterns.com/architecture-performance-and-games.html>. [Geopend 05 january 2026].
- [34] E. Evans, „Domain-Driven Design,” in *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Amsterdam, Pearson Benelux, 2003, p. temp.
- [35] R. C. Martin, Clean Architecture, Hoboken, New Jersey: Prentice Hall/Pearson, 2017, pp. 111-132.
- [36] L. W. S. L. O. Q. Yihan Duan, „Layered Architecture,” University of Waterloo, Waterloo, 2019.
- [37] J. May, „Enjoyable Game Architecture,” Chickensoft, 09 july 2025. [Online]. Available: <https://chickensoft.games/blog/game-architecture>. [Geopend 05 january 2026].
- [38] A. Cockburn, Hexagonal Architecture Explained, Salt Lake City, Utah: Humans and Technology Inc, 2005.
- [39] B. Mikoski, „Functional Unity Architecture: A Developer's Guide,” Medium, 29 october 2023. [Online]. Available: <https://medium.com/@bada/functional-unity-architecture-a-developers-guide-359e5111c5c2>. [Geopend 05 januari 2026].
- [40] T. M. Gokdemir, „Design Patterns for Unity Developers: Service Locator,” Medium, 30 october 2023. [Online]. Available: <https://medium.com/@taha.m.gokdemir/design-patterns-for-unity-developers-service-locator-124cd4628c43>. [Geopend 05 januari 2026].
- [41] M. Dealessandri, „What is the best game engine: is Unity right for you?,” gamesindustrybiz, 16 januari 2020. [Online]. Available: <https://www.gamesindustry.biz/what-is-the-best-game-engine-is-unity-the-right-game-engine-for-you>. [Geopend 26 november 2025].
- [42] M. Dealessandri, „What is the best game engine: is Godot right for you?,” gamesindustrybiz, 15 april 2020. [Online]. Available: <https://www.gamesindustry.biz/what-is-the-best-game-engine-is-godot-right-for-you>. [Geopend 25 november 2025].
- [43] M. Zechner, „libGDX 1.0 released,” badlogic games, 20 april 2014. [Online]. Available:

- <https://web.archive.org/web/20210213212631/https://www.badlogicgames.com/wordpress/?p=3412>. [Geopend 25 november 2025].
- [44] clickteam, „Clickteam Fusion 2.5,” clickteam, [Online]. Available: <https://www.clickteam.com/clickteam-fusion-2-5>. [Geopend 25 november 2025].
 - [45] ClickteamLLC, „Clickteam Fusion 2.5 Teaser Trailer,” ClickteamLLC, 27 november 2013. [Online]. Available: <https://youtu.be/XYQL1jL50qI>. [Geopend 25 november 2025].
 - [46] Photon, „Multiplayer Game Development Made Easy | Photon Engine,” Photon, [Online]. Available: <https://www.photonengine.com/>. [Geopend 25 november 2025].
 - [47] Epic Games, „Features - Unreal Engine,” Epic Games, [Online]. Available: <https://www.unrealengine.com/en-US/features>. [Geopend 25 november 2025].
 - [48] B. Sinclair, „Epic launches Unreal Engine 5,” gamesindustrybiz, 6 april 2022. [Online]. Available: <https://www.gamesindustry.biz/epic-launches-unreal-engine-5>. [Geopend 25 november 2025].
 - [49] Fandom, „Gameplay,” Fandom, 26 october 2018. [Online]. Available: <https://hearthstone.fandom.com/wiki/Gameplay>. [Geopend 05 january 2026].
 - [50] Yu-gi-oh!, „Yu-gi-oh! OFFICIAL RULEBOOK,” 27 july 2017. [Online]. Available: https://img.yugioh-card.com/en/downloads/rulebook/SD_RuleBook_EN_10.pdf. [Geopend 05 january 2026].
 - [51] A. Cimo, „How To Play Yu-Gi-Oh in 3 Minutes!,” Cinoooooooooooo, 17 august 2016. [Online]. Available: https://www.youtube.com/watch?v=55gsu3_36Fs. [Geopend 05 january 2026].
 - [52] Fandom, „Master Rules,” Fandom, 11 january 2012. [Online]. Available: https://yugioh.fandom.com/wiki/Master_Rules. [Geopend 05 january 2026].
 - [53] Fandom, „Level 5 Monster Cards,” 11 october 2008. [Online]. Available: https://yugioh.fandom.com/wiki/Level_5_Monster_Cards. [Geopend 05 january 2026].
 - [54] K. Telang, „slideshare,” november 2013. [Online]. Available: <https://www.slideshare.net/slideshow/game-art-bible-secret-sauce-to-making-great-game-art/28331088#19>. [Geopend 5 januari 2026].
 - [55] Splash Damage, „Internet Archive,” 16 april 2018. [Online]. Available: https://archive.org/details/vghf_design_doc_archive/dirty_bomb-game_design_document/page/n305/mode/1up. [Geopend 2 december 2025].
 - [56] J. French, „How to write a game design document (with examples),” 6 mei 2022. [Online]. Available: https://gamedevbeginner.com/how-to-write-a-game-design-document-with-examples/#how_to_make_a_design_document. [Geopend 2 december 2025].
 - [57] M. Zhang, „Magarena,” 9 april 2012. [Online]. Available: <https://github.com/magarena/magarena/tree/master/src/magic/model>. [Geopend 05 january 2026].
 - [58] Hanmac, „forge,” 05 augustus 2011. [Online]. Available: <https://github.com/Card-Forge/forge>. [Geopend 05 january 2026].

- [59] Microsoft, „The .NET Compiler Platform SDK,” Microsoft, 25 october 2024. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/>. [Geopend 05 january 2026].
- [60] M. Head, „C# Scripting with Roslyn,” Medium, 30 september 2023. [Online]. Available: https://medium.com/@Michael_Head/c-scripting-with-roslyn-7df86fdb2b26. [Geopend 05 january 2026].
- [61] A. Bahraminezhad, „Getting Started with Roslyn C# Scripting API,” ITNEXT, 3 March 2020. [Online]. Available: <https://itnext.io/getting-start-with-roslyn-c-scripting-api-d2ea10338d2b>. [Geopend 05 january 2026].

12 Bijlage

12.1 Kaarten

- Totaal aantal kaarten → 100 kaarten
- Aantal kaarten per class → 10 kaarten
- Neutraal → 50 kaarten
- Klasse specifiek → 50 kaarten

12.1.1 Neutral minions

12.1.1.1 1-Mana Minions

1. Rusted Tinkerling — 1 mana

200 ATK / 300 HP

A tiny worker drone.

Deathrattle: Restore 100 HP to your hero.

2. Brass Rat Scavenger — 1 mana

250 ATK / 200 HP

Ranged

When this attacks, **draw 1 card**

3. Gearshift Sprite — 1 mana

100 ATK / 400 HP

Back Row only

At end of turn, **give a random ally +50 HP**.

4. Steam-Piston Grub — 1 mana

300 ATK / 300 HP

Front Row

No abilities.

12.1.1.2 2-Mana minions

5. Ironclad Footman — 2 mana

500 ATK / 400 HP

Front Row

Gains **+100 HP** when placed in front row.

6. Copper-Weld Crossbowman — 2 mana

400 ATK / 500 HP

Ranged

7. Boilerplate Bully — 2 mana

600 ATK / 350 HP

Front Row – Taunt

8. Tock-Tock Saboteur — 2 mana

300 ATK / 450 HP

Battlecry: **Deal 200 damage** to a random enemy minion.

9. Alloy Sentry Drone — 2 mana

200 ATK / 700 HP

Back Row – Support

Adjacent allies gain **+50 HP**.

10. Wind-Up Scout Hawk — 2 mana

100 ATK / 300 HP

Ranged, Flying

Can target back-row minions even if front row isn't cleared.

12.1.1.3 3-Mana Minions

11. Steam Rail Bruiser — 3 mana

900 ATK / 700 HP

Front Row

Takes **–100 damage** from attacks.

12. Clockwork Sharpshooter — 3 mana

600 ATK / 600 HP

Ranged

Every other turn, an attack also inflicts damage to an adjacent minion

13. Gearwright Medic — 3 mana

200 ATK / 800 HP

Back Row – Support

Battlecry: **Heal a friendly minion for 400 HP**.

14. Ironbeak Reconstrux — 3 mana

700 ATK / 500 HP

Battlecry: **stuns** a minion.

15. Coppercoil Skirmisher — 3 mana

800 ATK / 600 HP

Front Row

16. Fog-Chamber Evader — 3 mana

500 ATK / 900 HP

Back Row

This card cannot be targeted unless it's the **only** back-row minion.

17. Rivetback Mulebot — 3 mana

400 ATK / 1200 HP

Front Row

Deathrattle: Draw 1 card

12.1.1.4 4-Mana Minions

18. Ironjaw Brawler — 4 mana

1200 ATK / 900 HP

Front Row

No abilities.

19. Smoke-Stack Sniper — 4 mana

900 ATK / 700 HP

Ranged – Sniper

Can always target back-row minions.

20. Voltwire Disruptor — 4 mana

600 ATK / 1000 HP

Battlecry: Enemy minions **lose 200 ATK** this turn.

21. Cogsmith Quartermaster — 4 mana

300 ATK / 1400 HP

Back Row – Support

Gives adjacent minions **+150 ATK**.

22. Scrap-Heap Guardian — 4 mana

800 ATK / 1500 HP

Front Row – Taunt

Large plated defender.

23. Gyro-Blade Skimmer — 4 mana

1100 ATK / 900 HP

Flying, Ranged

If placed in back row, gains **+200 ATK**.

12.1.1.5 5-Mana Minions

24. Arc-Driven Battlemech — 5 mana

1500 ATK / 1400 HP

Front Row

If this deals hero damage, **gain +200 ATK** permanently.

25. Smog-Belcher Tower — 5 mana

400 ATK / 2000 HP

Back Row

End of turn: **Deal 100 damage** to all enemy minions.

26. Gearforge Kinetic Knight — 5 mana

1800 ATK / 1000 HP

Front Row

Attacks deal **Splash 200 damage** to adjacent enemies.

27. Tincap Demolitionist — 5 mana

900 ATK / 1200 HP

Deathrattle: Deal **400 damage** to the minion that killed it.

28. Steamcall Mechanist — 5 mana

500 ATK / 1500 HP

Battlecry: Summon a **1-mana 300/300 SteamPiston Grub** next to it.

12.1.1.6 6-Mana Minions

29. Brass Titan Enforcer — 6 mana

2100 ATK / 1800 HP

Front Row – Taunt

This minion takes **50% damage** from ranged attacks.

30. Ironveil Artillery — 6 mana

1600 ATK / 1200 HP

Back Row – Siege

Can only attack **front row**, but deals **+400 damage** to it.

31. Cogborn Warlord — 6 mana

1700 ATK / 1700 HP

Front Row

All other front-row allies gain **+200 ATK**.

32. Storm-Coil Dynamo — 6 mana

0 ATK / 1900 HP

Back Row – Support

Playing a spell now restores 1 mana

12.1.1.7 7-Mana Minions

33. Clock-Titan Vanguard — 7 mana

2600 ATK / 2200 HP

Front Row

Piercing: Damage that exceeds target HP hits the minion behind.

34. Gaslamp Howitzer Crew — 7 mana

2000 ATK / 1600 HP

Back Row – Siege

Battlecry: Deal **600 damage** to a random enemy minion.

12.1.1.8 8-Mana Minions

35. Leviathan Gear-Colossus — 8 mana

3100 ATK / 2800 HP

Front Row

Cannot be moved from its row.

36. Ether-Plated Oracle Engine — 8 mana

0 ATK / 3500 HP

Back Row – Support

End of turn: Draw **2 cards**.

12.1.1.9 9-Mana Minions

37. Overwind Mammoth-Construct — 9 mana

4000 ATK / 3500 HP

Front Row

When this attacks, deal 300 damage to all enemies.

12.1.1.10 10-Mana Minions

38. The Great Steam Imperator — 10 mana

5000 ATK / 5000 HP

Front Row – Colossus

This card cannot be targeted by spells.

When summoned: **Stun all enemy minions for 1 turn.**

12.1.2 Neutral spells:

1. Boiler Burst — 2 mana

Deal **400 damage** to a minion.

2. Grease the Gears — 1 mana

Give a minion **+300 HP**.

3. Reinforced Plating — 3 mana

Give a front-row minion **+400 HP and Invincible** this turn.

4. Rapid Recalibration — 2 mana

Move one of your minions to the other row.

Draw 1 card.

5. Twin-Spark Volley — 4 mana

Deal **300 damage** to two targets.

6. Pressurized Slam — 5 mana

A friendly front-row minion attacks an enemy minion immediately.

7. Scrap to Steel — 3 mana

Destroy one of your minions.

Give its stats to another friendly minion.

8. Aether Vortex — 6 mana

Heal a minion for 1500HP but return it to its owner's hand

9. Steamfield Reinforcement — 4 mana

Gives 200 HP to every front-row minion

10. Machinist's Insight — 3 mana

Draw **two cards**.

11. Overclock Surge — 5 mana

Give a minion **+700 ATK** this turn.

12. Catastrophic Blowout — 7 mana

Deal **900 damage** to all enemies.

12.1.3 Class specific

12.1.3.1 Warrior

Heavy frontliners, durability, simple raw power

12.1.3.1.1 Minions

1. Steelplated Grunt — 2 mana

700 ATK / 700 HP

Front Row

No ability.

2. Iron Legion Shieldbearer — 3 mana

500 ATK / 1500 HP

Front Row – Taunt

Takes **–100 damage** from all sources.

3. Rivetforge Halberdier — 4 mana

1300 ATK / 900 HP

Front Row

4. Brassbreaker Elite — 5 mana

1900 ATK / 1600 HP

Front Row

Whenever this minion attacks, gain **+100 HP**.

5. Bunker-Tower Marauder — 6 mana

1700 ATK / 2500 HP

Front Row – Taunt

Adjacent allies gain **+200 HP**.

6. Iron Legion Juggernaut — 8 mana

3500 ATK / 3000 HP

Front Row

Excess damage hits the hero directly.

12.1.3.1.2 Spells

1. Reinforced Alloys — 2 mana

Give a friendly minion **+500 HP**.

2. Crank-Up Assault — 3 mana

A friendly minion gains **+600 ATK** this turn.

3. Warsteel Formation — 5 mana

Give all front-row allies **+200 ATK and +200 HP**.

4. Titanwall Clash — 7 mana

Your front row attacks all enemy front-row minions once.

12.1.3.2 Mage

Spells, elemental automatons, buffs/debuffs, back-row casters

12.1.3.2.1 Minions

1. Aether Spark Familiar — 2 mana

200 ATK / 600 HP

Back Row – Caster

Your spells deal **+100 damage**.

2. Cloudcore Channeler — 3 mana

300 ATK / 900 HP

Back Row – Caster

Battlecry: Draw 1 card.

3. Emberforge Golem — 4 mana

1200 ATK / 1000 HP

Front Row

Battlecry: Deal **200 damage** to an enemy minion.

4. Arcane Condenser Node — 4 mana

0 ATK / 1600 HP

Back Row – Support

When you cast a spell, give a minion **+100 HP**.

5. Stormcoil Invoker — 6 mana

1000 ATK / 1600 HP

Back Row – Caster

All healing to this card is **doubled**

6. Aether Titan of Ruin — 8 mana

2400 ATK / 2500 HP

Back Row – Caster

Battlecry: Deal **800 damage** split among enemies

12.1.3.2 Spells

1. Voltaic Bolt — 1 mana

Deal **200 damage** to a minion.

2. Aether Reinforcement — 3 mana

Give a minion **+400 HP and +300 ATK**.

3. Time-Slip Mirrorfield — 4 mana

Copy an the effect of the other players card in the same spot

4. Cataclysmic Surge — 7 mana

Deal **700 damage** to all enemy minions.

12.1.3.3 Priest

Healing, protection, buffs, radiant constructs

12.1.3.3.1 Minions

1. Brasslight Acolyte — 2 mana

200 ATK / 700 HP

Back Row

End of turn: Heal a friendly minion for **200 HP**.

2. Halo Mechanist — 3 mana

400 ATK / 800 HP

Battlecry: Give a minion **+300 HP**.

3. Sacred Lantern Warden — 3 mana

350 ATK / 1100 HP

Front Row – Taunt

First time this would die, heal it to **300 HP**.

4. Beam-Forge Guardian — 5 mana

900 ATK / 1700 HP

Front Row

All heals add the same amount to your **attack**

5. Radiant Servo-Seraph — 6 mana

1000 ATK / 1800 HP

Back Row

Your healing effects restore **+150 HP** extra.

6. Brasslight Ascendant — 8 mana

2000 ATK / 3000 HP

Front Row

Battlecry: Heal all allies for **400 HP**.

12.1.3.3.2 Spells

1. Purifying Beam — 2 mana

Heal a minion for **500 HP**.

2. Sanctuary Field — 3 mana

Give a minion **Invincible** for a turn and **+600 HP**.

3. Radiance Reversal — 5 mana

Heal all friendly minions for **200 HP**.

Deal that amount to a random enemy.

4. Blessing of the Brasslight — 7 mana

Give all allies **+400 HP**.

12.1.3.4 Hunter

Traps, beast, ranged threats, punishing attacks

12.1.3.4.1 Minions

1. Scraptooth Cub — 1 mana

300 ATK / 300 HP

Front Row

No ability.

2. Wildsteel Archer — 2 mana

500 ATK / 400 HP

Ranged

Can attack the back row.

3. Ferroclaw Tracker — 3 mana

700 ATK / 700 HP

Front Row

Battlecry: When this card is deployed, restore 3 mana.

4. Hunter's Trapwright — 4 mana

300 ATK / 1000 HP

Back Row

When this minion is attacked, deal **200 damage** to the attacker.

5. Stormfang Direwolf — 6 mana

1900 ATK / 1200 HP

Front Row – Beast

Whenever this attacks, deal **200 damage** to a random enemy.

6. Apex Gear-Behemoth — 8 mana

2800 ATK / 2200 HP

Front Row

If this kills a minion, it deals **1000ATK** to a random enemy (cannot trigger twice)

12.1.3.4.2 Spells

1. Razorjaw Trap — 2 mana

Secret: When an enemy attacks a friendly minion, deal **600 damage** to the attacker.

2. Sootmark Shot — 3 mana

Deal **400 damage** to a minion.

If it dies, **draw a card**.

3. Wildcall Assembly — 5 mana

Summon two **3-mana 700/700 Beastlings**.

4. Primeval Howl — 7 mana

Give all your minions **+300 ATK** and they can attack **back-row enemies** this turn.

12.1.3.5 Rogue

Trickery, stealth, swapping, theft, illusions

12.1.3.5.1 Minions

1. Gearshift Cutpurse — 2 mana

500 ATK / 300 HP

Front Row – Combo

Battlecry: **steal 1 card** from opponent's discard pile.

2. Mistcoil Infiltrator — 3 mana

300 ATK / 800 HP

Back Row – Stealth

Cannot be targeted until it attacks.

3. Smokewire Saboteur — 3 mana

600 ATK / 600 HP

Front Row?

Battlecry: A random enemy minion loses 300 ATK this turn.

4. Rustveil Doppelganger — 4 mana

700 ATK / 900 HP

Front Row

Copies the effect of the enemy in front of it.

5. Shadowclaw Assassin — 5 mana

1700 ATK / 700 HP

Front Row – Strike

Can attack **back-row** minions directly.

6. Master of Disguise — 7 mana

1200 ATK / 1900 HP

Back Row – Illusion

Battlecry: Choose a minion. It becomes a **1/1 Illusion** until end of next turn. (Currently Not Implemented)

12.1.3.5.2 Spells

1. Smoke Bomb — 1 mana

Makes an ally **invincible for a turn**

2. Cogblade Stab — 2 mana

Deal **300 damage**.

If the target dies, deal **200 damage** to another random enemy.

3. Stolen Blueprint — 4 mana

Copy a random card from your opponent's hand.

4. Mechanical Mayhem — 6 mana

Withdraw one of your enemies cards to their hand.

12.1.4 Rarity

How rare is a card and how many of the same card can a person have per deck

12.1.4.1 Neutral

12.1.4.1.1 Minions

12.1.4.1.1.1 Common → Up to 3 copies

Basic troops, low-level spells, simple effects

- Rusted Tinkerling
Brass Rat Scavenger
- Gearshift Sprite
- Steam-Piston Grub
- Ironclad Footman
- Copper-Weld Crossbowman
- Tock-Tock Saboteur
- Alloy Sentry Drone
- Wind-Up Scout Hawk
- Steam Rail Bruiser
- Rivetback Mulebot
- Ironjaw Brawler
- Scrap-Heap Guardian
- Smog-Belcher Tower
- Ironveil Artillery
- Overwind Mammoth-Construct

12.1.4.1.1.2 Rare → Up to 2 copies

Enhanced stats or mechanics, solid utility

- Boilerplate Bully
- Clockwork Sharpshooter
- Gearwright Medic
- Ironbeak Reconstrux
- Coppercoil Skirmisher

- Fog-Chamber Evader
- Smoke-Stack Sniper
- Voltwire Disruptor
- Cogsmith Quartermaster
- Gyro-Blade Skimmer
- Tincap Demolitionist
- Storm-Coil Dynamo
- Gaslamp Howitzer Crew

12.1.4.1.1.3 *Epic → up to 2 copies*

Powerful effects, strong synergies, major spells

- Arc-Driven Battlemech
- Gearforge Kinetic Knight
- Brass Titan Enforcer
- Cogborn Warlord
- Clock-Titan Vanguard
- Leviathan Gear-Colossus
- Ether-Plated Oracle Engine

12.1.4.1.1.4 *Legendary → 1 copy*

Unique minions, signature spells, iconic class cards

- The Great Steam Imperator
- Overwind Mammoth-Construc

12.1.4.1.2 Spells

12.1.4.1.2.1 *Common*

- Boiler Burst
- Grease the Gears
- Steamfield Reinforcement
- Machinist's Insight
- Twin-Spark Volley

12.1.4.1.2.2 *Rare*

- Reinforced Plating
- Rapid Recalibration
- Pressurized Slam
- Scrap to Steel

12.1.4.1.2.3 *Epic*

- Aether Vortex
- Overclock Surge

12.1.4.1.2.4 *Legendary*

- Catastrophic Blowout

12.1.4.2 Class specific

12.1.4.2.1 Warrior class

Common

- Steelplated Grunt
- Iron Legion Shieldbearer
- Rivetforge Halberdier
- Crank-Up Assault (spell)

Rare

- Brassbreaker Elite
- Reinforced Alloys (spell)
- Warsteel Formation (spell)

Epic

- Bunker-Tower Marauder
- Titanwall Clash (spell)

Legendary

- Iron Legion Juggernaut

12.1.4.2.2 Mage

Common

- Aether Spark Familiar
- Cloudcore Channeler
- Voltaic Bolt (spell)
- Aether Reinforcement (spell)

Rare

- Emberforge Golem
- Arcane Condenser Node
- Time-Slip Mirrorfield (spell)

Epic

- Stromcoil Invoker
- Cataclysmic Surge (spell)

Legendary

- Aether Titan of Ruin

12.1.4.2.3 Priest

Common

- Brasslight Acolyte
- Halo Mechanist
- Purifying Beam (spell)
- Sanctuary Field (spell)

Rare

- Sacred Lantern Warden
- Beam-Forge Guardian
- Radiance Reversal (spell)

Epic

- Radiant Servo-Seraph
- Blessing of the Brasslight (spell)

Legendary

- Brasslight Ascendant

12.1.4.2.4 Hunter

Common

- Scraptooth Cub
- Wildsteel Archer
- Razorjaw Trap (spell)
- Sootmark Shot (spell)

Rare

- Ferroclaw Tracker
- Hunter's Trapwright
- Wildcall Assembly (spell)

Epic

- Stormfang Direwolf
- Primeval Howl (spell)

Legendary

- Apex Gear-Behemoth

12.1.4.2.5 Rogue

Common

- Gearshift Cutpurse
- Mistcoil Infiltrator
- Smoke Bomb (spell)
- Cogblade Stab (spell)

Rare

- Smokewire Saboteur
- Rustveil Doppelganger
- Stolen Blueprint (spell)

Epic

- Shadowclaw Assassin
- Mechanical Mayhem (spell)

Legendary

- Master of Disguise